Contents

Instructions for Students

1. The students are expected to attend all the classes (100%) in this courses. For circumstances beyond their control, like medical reasons etc., they can miss classes provided missed classes do not exceed 20% of all classes. Failing which, students are allowed to appear in the final examination of the course in which attendance requirement is not fulfilled.
2. Students should bring their text books to the lab, so that they can refer to theory and diagrams whenever required.
3. The assessment sheet at the end of every lab looks like this:

Name: _____
Reg. No. : _____
Date of Lab: _____

	Excellent	**Very Good**	**Good**	**Satisfactory**	**Poor**	**Score**
	2.0	1.5	1.0	0.5	0	
Apparatus Handling	Can independently setup, operate and handle the apparatus	Can setup and handle the apparatus with minimal help	Can setup and handle the apparatus with some help	Cannot setup apparatus according to design but knows how to handle apparatus	Cannot setup or handle the apparatus	
Problem Solution	Complete understanding of the experiment to be performed	Unable to link the problem and solution but understands both individually	Solution is partially interpreted	Can assemble but cannot interpret the solution	Cannot understand and interpret the solution	
Teamwork	Performs experiment and assists other group members in experiment	Observes carefully, performs experiment himself in group	Participate in group work but do not lead the group	Only observes and never performs experiment himself in group	Does not participate in lab group	
Output Accuracy	Most efficient and most accurate measurement and results	Measurements are correctly made and results are presented accurately	Measurements are made correctly and results are not presented accurately	Measurements are made and results are not presented accurately	Measurements are not made and results are not presented accurately	
Viva	Accurate knowledge of every aspect of the lab	Partial practical and partial background knowledge	Have confidence of the practical knowledge but no background knowledge	Unable to clarify the results of the lab	Unable to answer any question	
Total Score						/10

Lab – 1: Introduction to SAP – I

SAP (Simple-As-Possible) is a computer designed for beginners. The main purpose of SAP is to introduce all the crucial concepts behind computer operations. SAP- 1 is the first stage in the evolution toward modern computers. Although primitive, SAP is considered to be a big step for a beginner.

1.1 Architecture

Figure 1-1 shows the architecture of SAP-1 i.e. a bus-organized computer.
All register outputs to the 8 bit W-bus are three states; this allows orderly transfer of data. All other register outputs are of two state; these outputs continuously drive the boxes they are connected to. The layout of Fig. 1-1 emphasizes on the registers used in SAP- 1. For this reason, no attempt has been made to keep all control circuits in one block called the control unit, all input-output circuits in another block called the I/O unit, etc.

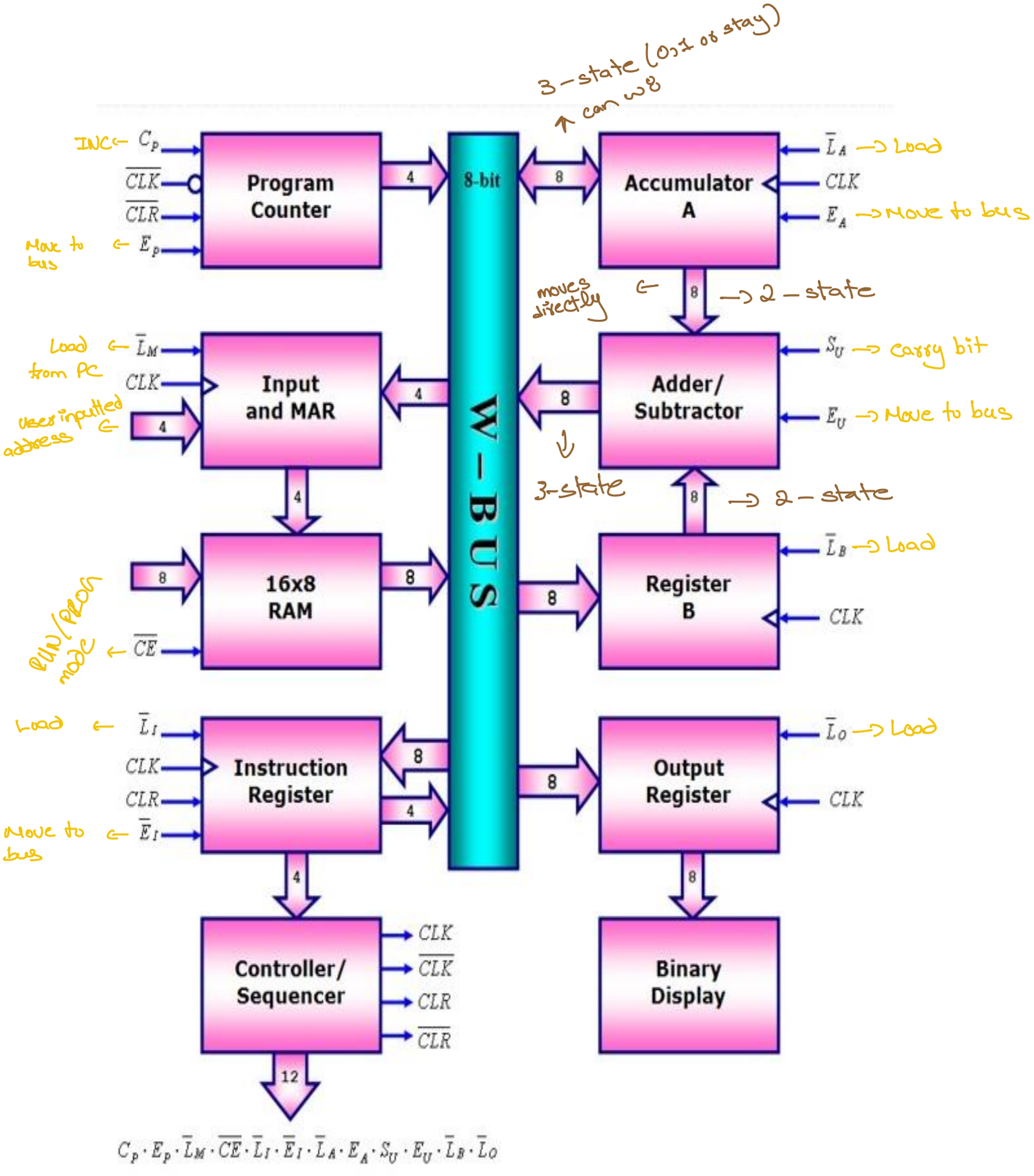


Figure 1-1: Architecture for SAP – 1 Computer

1.2 Program Counter

1. The program is stored at the beginning of the memory with the first instruction at binary address 0000, the second instruction at address 0001, the third at address 0010, and so on.
2. The program counter counts from 0000 to 1111.
3. Its job is to send to the memory the address of the next instruction to be fetched and executed. It does this as follows.
    a. The program counter is reset to 0000 before each program execution.
    b. When the execution process begins, the program counter sends address 0000 to the memory.
    c. The program counter is then incremented to 0001.
    d. After the first instruction is fetched and executed, the program counter sends address 0001 to the memory.
    e. Again the program counter is incremented to 0010.
    f. After the second instruction is fetched and executed, the program counter sends address 0010 to the memory and so on until all the instructions of the program are executed.
4. In this way, the program counter keeps track of the next instruction to be fetched and executed.
5. The program counter is like someone pointing a finger at a list of instructions, saying do this first, do this second do this third, etc.
6. This is why the program counter is sometimes called a pointer as it points to an address in memory where something important is being stored

1.3 Input and MAR

1. Below the program counter is the input and (Memory Address Register) MAR block.
2. This block acts as an address and data switch registers, i.e. when data is required it acts as a data register and if an address is required it acts as an address register.
3. These switch registers, which are part of the input unit, allows to send 4 address bits and 8 data bits to the RAM (Random Access Memory).
4. As you recall, instruction and data words are written into the RAM before a program execution
5. The memory address register (MAR) is part of the SAP-1 memory.
6. During program execution, the address in the program counter is latched into the MAR.
7. A bit later, the MAR applies this 4-bit address to the RAM, where a read operation is performed.

1.4 RAM (Random Access Memory)

1. The RAM is 16x8 static TTL (Transistor–transistor logic) RAM.
2. It can be programmed by means of address and data switch registers.
3. During a program execution, the RAM receives a 4-bit address from MAR and a read operation is performed.
4. In this way, an instruction or data word stored in RAM is placed on the W bus for use in some other part of the computer.

1.5 Instruction Register

1. The instruction register is part of the processor.
2. To fetch an instruction from the memory the computer does a memory read operation.
3. This places the contents of the addressed memory location on the W bus.
4. At the same time, the instruction register is set up for loading on the next positive clock edge.
5. The contents of the instruction register are split into two nibbles.
6. The upper nibble is a two-state output that goes directly to the block labeled "Controller-sequencer".
7. The lower nibble is a three-state output that is read onto the W bus when needed.

1.6 Controller-Sequencer

1. The lower left block contains the controller-sequencer.

Before each computer run, a CLR signal is sent, to the program counter and a CLR signal to the instruction register.

1. This resets the program counter to 0000 and wipes out the last instruction in instruction register.
2. A clock signal CLK is sent to all buffer registers; this synchronizes the operation of the computer, ensuring that things happen when they are supposed to happen. In other words, all register transfers occur on the positive edge of a common CLK signal.
3. Notice that a CLK signal also goes to the program counter.
4. The 12 bits that come out of the controller-sequencer form a word controlling the rest of the computer (like a supervisor telling others what to do.) The 12 wires carrying the control word are called the control bus.
5. The control word has the format of

CON= $C_P \quad E_P \quad \overline{L_M} \quad \overline{C_E} \quad \overline{L_1} \quad \overline{E_1} \quad \overline{L_A} \quad E_A \quad S_U \quad E_U \quad \overline{L_B} \quad \overline{L_O}$

7. This word determines how the registers will react to the next positive CLK edge. For instance, a high EP and a low. The acronyms for the CON word stands for
   1) C_P=control program counter
   2) E_P= Enable program
   3) L_M=load memory
   4) C_E=control enable
   5) L_1= load first element
   6) E_1= enable first element
   7) L_A =load accumulator
   8) E_A= enable accumulator
   9) S_U=perform the operation of sum(S_U is low)/difference (S_U is high)
   10) E_U=enable the result
   11) L_B=load register B
   12) L_O=load output register
8. LM' mean that the contents of the program counter are latched into the MAR on the next positive clock edge.
9. As another example, a low CE' and a low LA' mean that the addressed RAM word will be transferred to the accumulator on the next positive clock edge.

1.7 Accumulator

1. The accumulator (A) is a buffer register that stores intermediate answers during a program execution In Fig. 1-1, the accumulator has two outputs.
2. The two-state output goes directly to the Adder-Subtractor.
3. The three-state output goes to the W bus.
4. Therefore, the 8-bit accumulator word continuously drives the Adder-Subtractor. The same word appears on the W bus when E_A is high.

1.8 The Adder-Subtractor

1. SAP-1 uses a 2's-complement Adder-Subtractor. When Su is low in Fig. 10-1, the sum out of the Adder-Subtractor is an addition
   When Su is high, the difference appears:
   (Recall that the 2's complement is equivalent to a decimal sign change.)
2. The Adder-Subtractor is asynchronous (un-clocked); this means that its contents can change as soon as the input words change.
3. When is high, these contents appear on the W bus.

1.9 B Register

1. The B register is another buffer register.
2. It is used in arithmetic operations. A low LB' and positive clock edge load the word on the W bus into the B register.
3. The two-state output of the B register drives the Adder-Subtractor, supplying the number to be added or subtracted from the contents of the accumulator.

1.10 Output Register

1. At the end of a computer run, the accumulator contains the answer to the problem being solved.
2. At this point, we need to transfer the answer to the outside world.
3. This is where the output register is used. When EA is high and L0' is low, the next positive clock edge loads the accumulator word into the output register.
4. The output register is often called an output port because processed data can leave the computer through this register.
5. In microcomputers the output ports are connected to interface circuits that drive peripheral devices like printers, cathode-ray tubes, teletypewriters, and so forth. (An interface circuit prepares the data to drive each device.)

1.11 Binary Display

1. The binary display is a row of eight light-emitting diodes (LEDs).
2. Because each LED connects to one flip-flop of the output port, the binary display shows us the contents of the output port.
3. Therefore, after we've transferred an answer from the accumulator to the output port, we can see the answer in binary form.

1.12 Instruction Set

1. A computer is a useless pile of hardware until someone programs it.
2. This means loading step-by-step instructions into the memory before the start of a program execution
3. The SAP-I instruction set consist on five instructions as given below;
   (a) LDA (Load accumulator)
   (b) ADD (Addition)
   (c) SUB (Subtraction)
   (d) OUT (display binary output)
   (e) HLT (stop program execution)

Lab – 2: Design a Clock Circuit for SAP-I

2.1 Objective

To build up a clock circuit that generates **Clear** and the **Clock** signal, that derives the Control Unit of SAP-1 computer.

2.2 Components Required

a) Quad two input NAND gate IC, 74LS 00…3
b) Quad three input NAND gate IC, 74LS10…1
c) Hex Inverter IC, 74LS04…1
d) 555 Timer IC…1
e) Dual J-K Flip Flop 74LS107… 1
f) Two Push button switches
g) Connecting wires

2.3 Clock Circuit

The clock circuit is shown in the figure 2.1 which consists of mainly a 555 Timer and a J-K flip flop. 555 Timer produces a 2-kHz clock signal with a 75% duty cycle at its output. The J-K flip flop divides the signal down to 1-kHz and also produces a 50% duty cycle.

Clock Buffers, which are two inverters, are used to produce the final clock signal, one CLK and other inverted CLK' signal. These are used here so that the clock signal being generated is able to drive low-power Schottky TTL loads.

Clear- Start De-bouncer produces two outputs, CLR for the Instruction Register and the inverted CLR' for the Program Counter and Ring Counter. S5 is a push button switch. When depressed, it goes to the clear position generating a high CLR and a low CLR'. When S5 is released, it returns to the START position, producing a low CLR and a high CLR'.

SAP-1 runs in either of the two modes, Manual or Automatic. In Manual mode you press and release S6 to generate one clock pulse. When S6 is depressed, CLK is high; when released CLK is low. In other words, the single-step de-bouncer generates T states one at a time as you press and release the button. This allows you to step through the different T states while troubleshooting or debugging.

Switch S7 is a single pole double throw SPDT switch that can remain in either the manual or the AUTO position. When in Manual the single-step button is active. When in Auto mode, the computer runs automatically. Two of the NAND gates are used to de-bounce the MANUAL-AUTO switch.

2.4 Procedure

1. Connect the circuit as shown in the circuit diagram i.e. Figure 2-1.
2. Check the signals with the oscilloscope and get your work verified by your Lab instructor.
3. You will be graded for this Lab on the output and functionality of your circuit.
4. For connections refer to the Pin configurations of the integrated circuit.

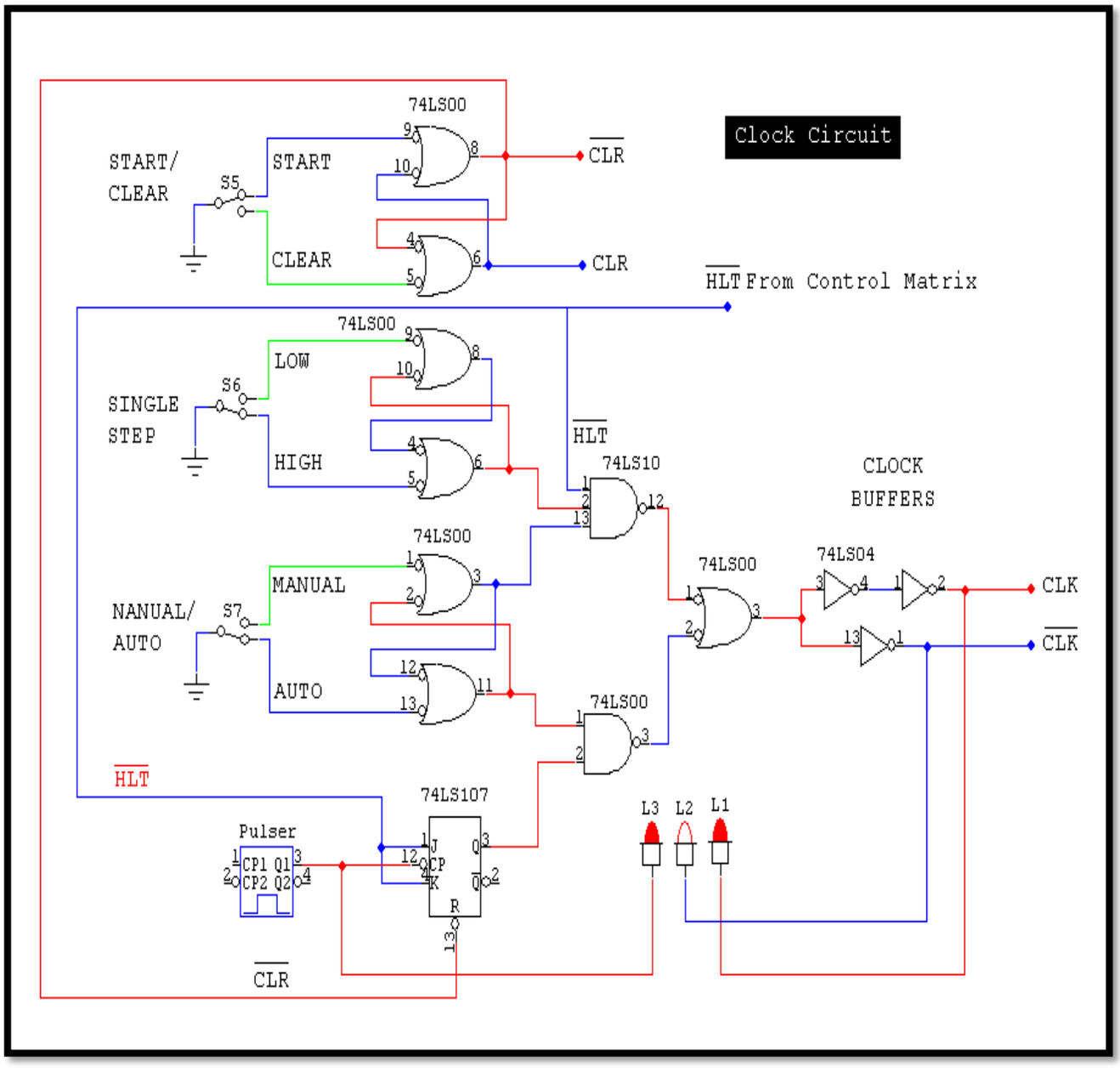


Figure 2-1: Clock Circuit for SAP – 1

Pin configurations of ICS

a) 74LS 00

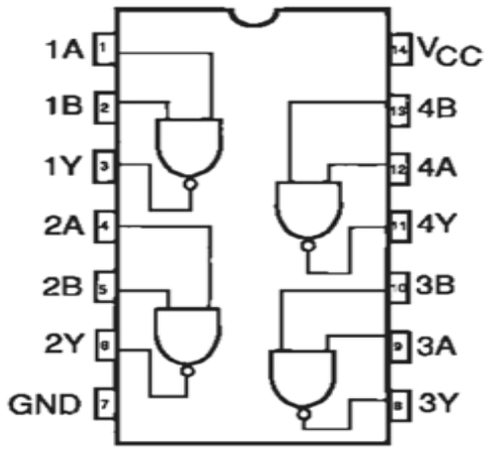


Figure 2-2: circuit diagram of 74LS00

b) 74LS10

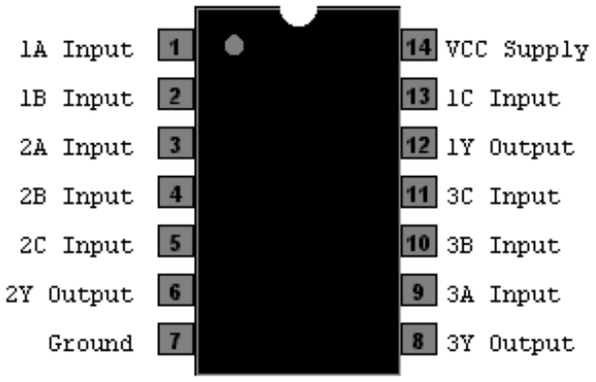


Figure 2-2: circuit diagram of 74LS10

c) 74LS04

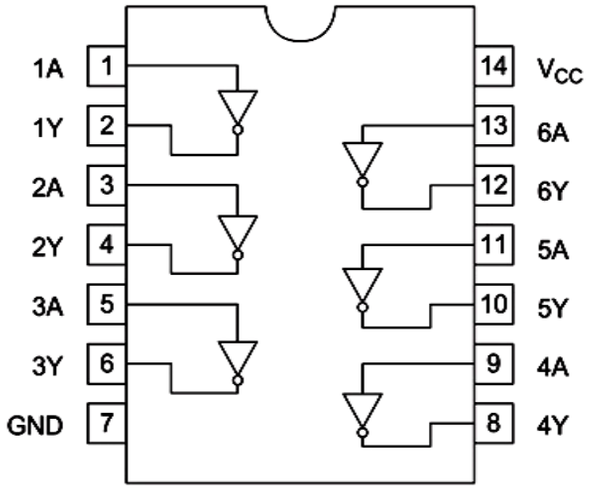


Figure 2-2: circuit diagram of 74LS04

d) 555 Timer IC

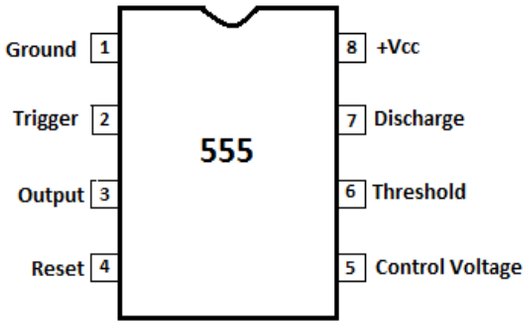


Figure 2-2: circuit diagram of 555 Timer

e) Dual J-K Flip Flop 74LS107

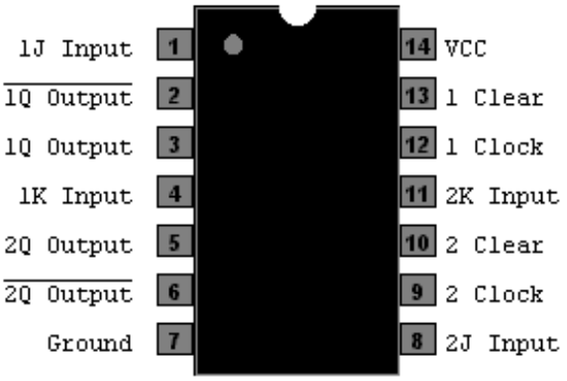


Figure 2-2: circuit diagram of Dual J-K Flip Flop 74LS107

Lab – 3: Program Counter, MAR, and 2-1 MUX

3.1 Objectives

To make the circuit of program counter for SAP-1 that generates a 4-bit address code of memory, Input MAR and 2 to 1 Multiplexer

3.2 Components Required:

a) Dual JK Flip Flop IC, 74LS107 … 2
b) Quad three state switch IC, 74LS126 …1
c) 4 bit D-type Register, 74LS173 … 1
d) Quad 2-to-1 Data Selector/ Multiplexer IC, 74LS157 … 1
e) Connecting wires

3.3 What is Program Counter?

The program counter counts according to the available RAM-memory. Its job is to send memory the address of the next instruction to be fetched and executed. Program counter is also called pointer, as it works by pointing towards a list of instructions stored at different addresses in the memory, saying do this first, do this second and so on.

3.4 MAR (Memory Address Register)

Chip C4, is a 74LS173, 4 bit buffer register. It serves as the MAR. Notice that pins 1 and 2 are grounded; this converts the three-state output to a two-state output. In other words, the output of the MAR is not connected to the W bus, and so there's no need to use the three-state output.

3.5 2-to-1 Nibble Multiplexer (Input unit)

Chip C5 is a 74LS157 2-to-1 nibble multiplexer. The left nibble (pins 14, 11, 5 & 2) come from the address switch register S1 (these are just 4 DIP switches of your trainer and will be used to manually program your RAM). The right nibble (pins 13, 10, 6, 3) comes from the MAR. The RUN-PROG switch S2 selects the nibble to reach to the output of C5. When S2 is in the PROG position (i.e. when it is low), the nibble out of the address switch register is selected. On the other hand, when S2 is in the RUN position (i.e. when it is high), the output of the MAR is selected.

S2 = 1 (RUN)
Output of MAR

S2 = 0 (PROGRAM)
output of S1 (users input)

3.6 Procedure

Connect the circuit as shown in the diagram.

Clear	Clk	Data Enable		Data	Output
		G1 G'1	G'2	D	Q
1	X	X	X	X	0
0	0	X	X	X	Q_o
0	POS	1	X	X	Q_o
0	POS	X	1	X	Q_0
0	POS	0	0	0	0
0	POS	0	0	1	1

Table 3-1: Truth table for 74LS173

Strobe G'	Select A'/B	A	B	Output Y
1	X	X	X	0
0	0	0	X	0
0	0	1	X	1
0	1	X	0	0
0	1	X	1	1

Table 3-2: Truth Table for 74LS157

3.6.1 How to check the output:

Connect CP, EP, CLR' to logic level 1 and LM' to logic level 0. Connect CLK' and CLK to the output of push button of trainer board (A AND A' OR B AND B').

Now the output of the 74LS157 will be A0, A1, A2 and A3 when Run/prog switch is connected to logic 0, and the output of 74LS157 will be a 4 bit counter that will count from 0 to 15 on each clock pulse when Run/prog switch is connected to logic level 1.

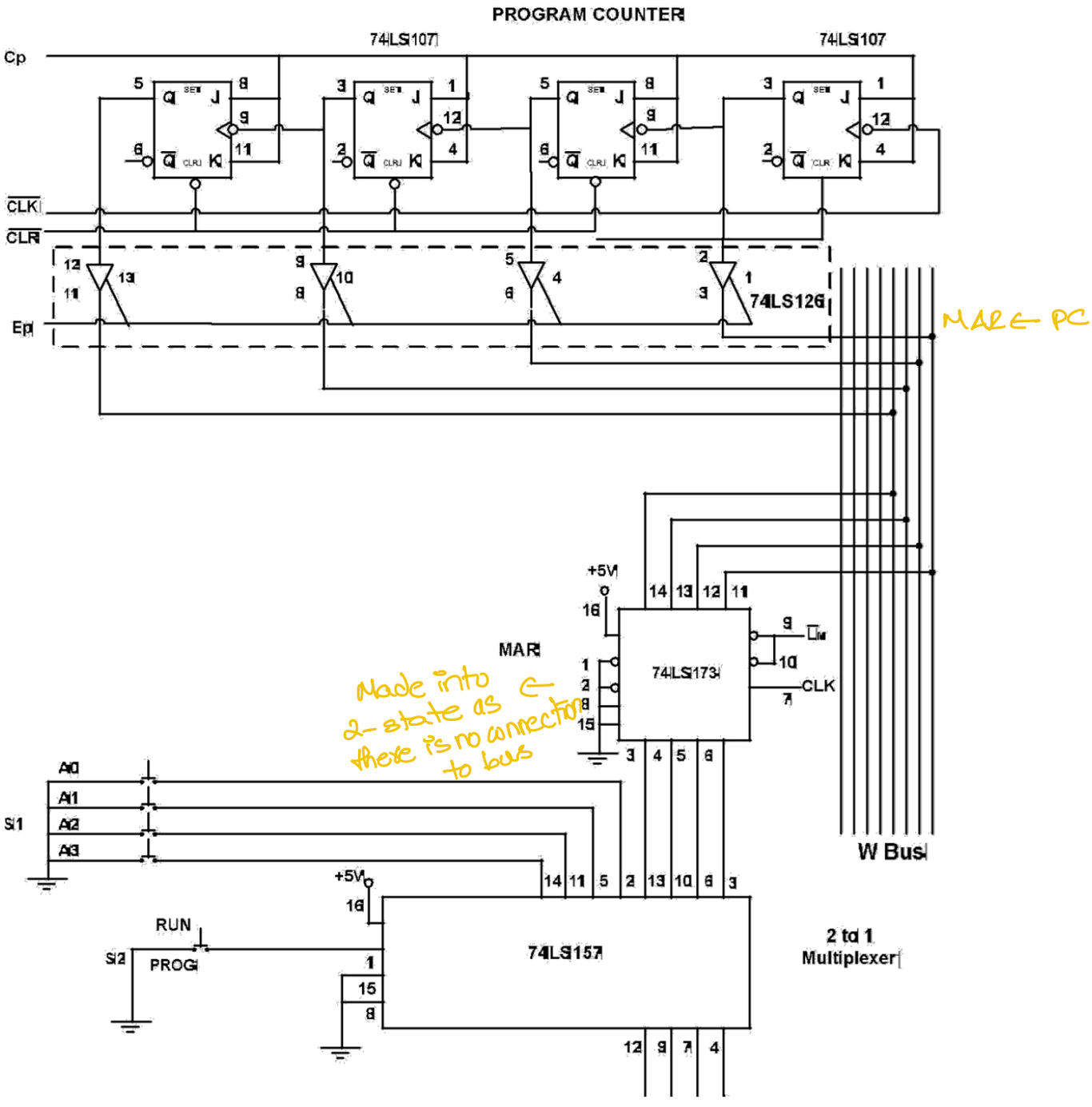


Figure 3-1: Program Counter and MAR

Pin configuration

a) 74LS107

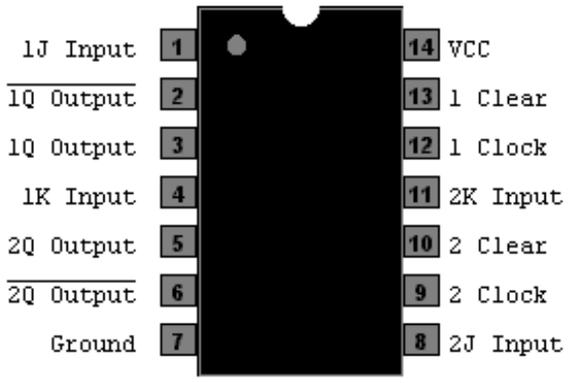


Figure 3-2: pin configuration for 74LS 107

b) 74LS126

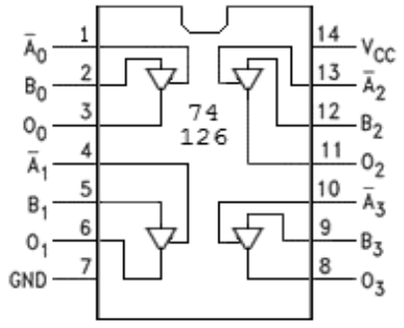


Figure 3-3: pin configuration for 74LS 126

c) 74LS173

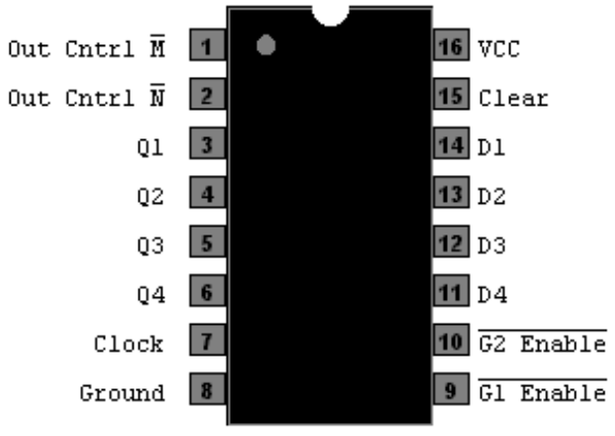


Figure 3-3: pin configuration for 74LS 173

d) 74LS157

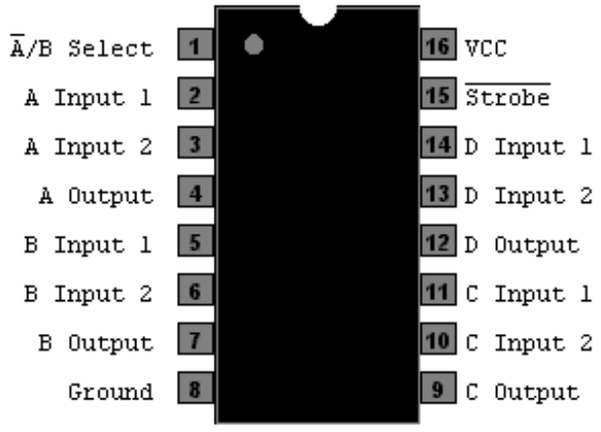


Figure 3-4: pin configuration for 74LS 157

Lab – 4: 16x8 RAM & Instruction Register

4.1 Objectives

To make the circuit of 16x8 RAM & Instruction-Register.

How to put information in memory?
1. Put S2 in PROM, CE is 0
2. Use address switch to choose locatio
   Use data switch to choose what data
3. Press WE to save it
4. Put S2 in RUN

4.2 Components Required

16x4, 74189…….2

4 bit D-type Register IC, 74LS173 … 2

Connecting Wires

4.3 16x8 RAM

The chips used here are 74189s.

Each chip is a 16 X 4 static RAM.

Together, they give us a 16 X 8 read-write memory.

S3 is a data switch register (8-bit), and S4 is a read- write switch (a push button switch).

To program the memory, S2 is put in the PROG position, this take the CE input low (pin 2).

The address and data switches are than set to the correct address and data words.

A momentary push of the read –write switch takes WE' low (pin3) and loads the memory.

After the program and data are in memory, the RUN-PROG switch (S2) is put in the run position in preparation for the program execution

4.4 Instruction Register

The chips used are 74LS173s.

Each chip is a 4-bit three-state buffer register. The two chips are the instruction-register.

Grounding pins 1 and 2 of C8 convert the three state output to a two-state output, I7, I6, I5 and I4 this nibble goes to the instruction decoder in the controller sequencer. Signal E1 controls the output of C9, the lower nibble in the instruction register.

When E1 is low, this nibble is placed on word bus.

4.5 Procedure

Connect the circuit as shown in the fig provided.

4.6 What should be the outcome?

As this lab is a continuation of previous one so by keeping the switch of Multiplexer on PROG mode address the RAM location through switches A0 to A3, and then write some inverted instruction by keeping the switches of RAM on Write & PROG mode.

Afterwards try to read the instruction by keeping the switch of Multiplexer on RUN mode and using the higher nibble for addressing the specific location, and keeping the switches of RAM on READ & RUN mode.

Instruction register should be able to read instruction from RAM, when LI' is active and

Keep Mux on PROM to get user's address, RAM on PROM & write to save instruction

Keep Mux on RUN mode to get MAR address RAM on RUN & read mode to get instruction on bus.

by making the LE' pin active the upper nibble (op code) should be ready for control matrix circuit and lower nibble (memory address) should be ready to be fed to the MAR.

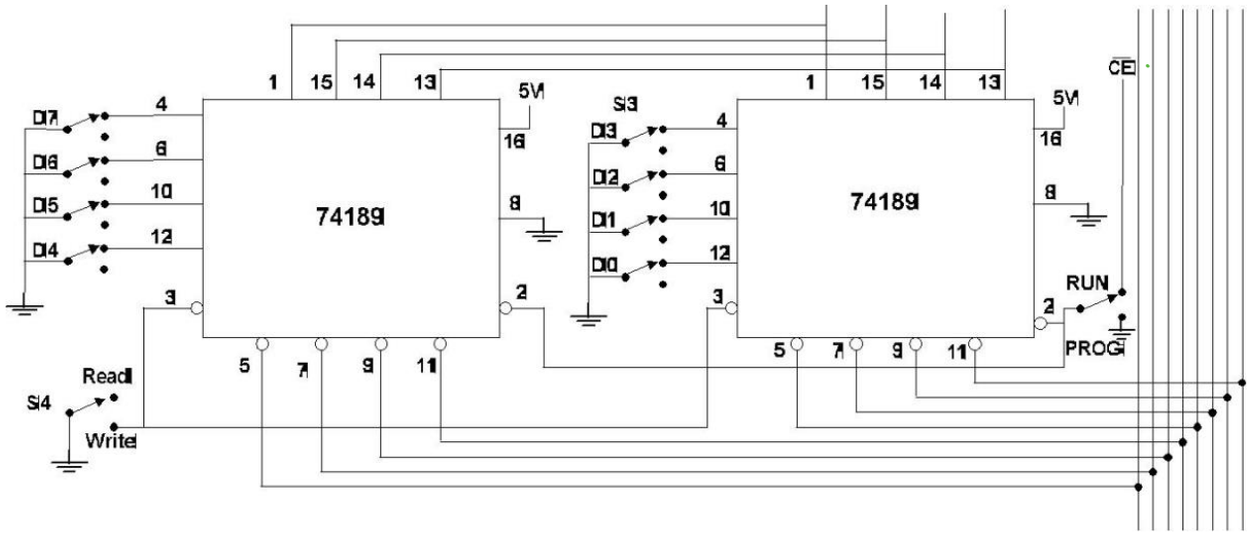


Figure 4-1: 16x8 RAM

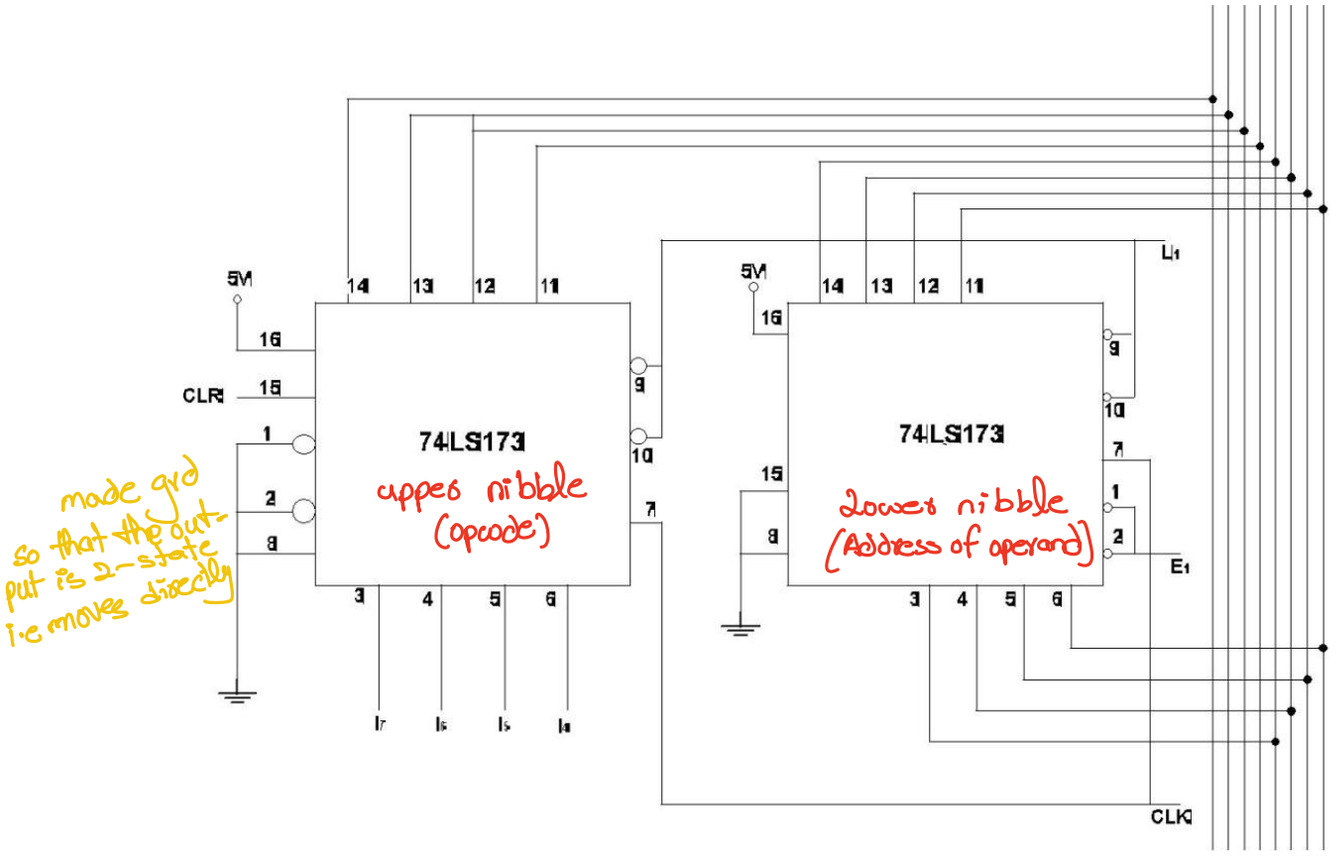


Figure 4-2: Instruction Register

Pin Configuration

a) 74189

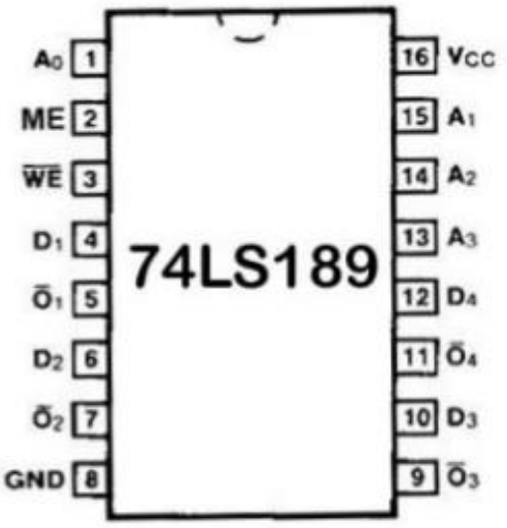


Figure 4-3: pin configuration for IC 74lS189

b) 74ls173

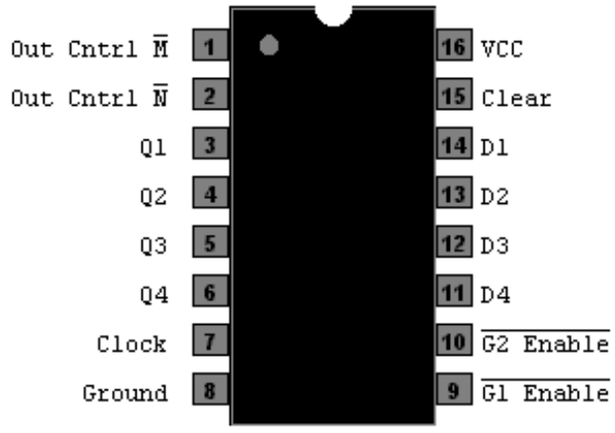


Figure 4-4: pin configuration for IC 74lS173

Lab – 5: Instruction Decoder, Ring Counter & Control Matrix

5.1 Objective

To build up the circuit of the Control Unit of SAP-1 computer including the instruction Decoder, the Ring Counter and the Control Matrix this generates the Control Word for SAP-1.

5.2 Components Required

Quad two input Nand gate IC, 74LS00 … 6
Tri three input Nand gate lC, 74LS10 … 1
Dual four input Nand gate IC, 74LS20 … 5
Hex Inverter IC, 74LS04 … 5
Dual J-K Flip Flop IC, 74LS107 … 3
Connecting wires

5.3 SAP-1 Instruction Set

Sap-1 computer has five instructions set. This instruction set is a list of basic operations the computer can perform. The instructions are:

1. LDA
2. ADD
3. SUB
4. OUT
5. HLT

5.3.1 Mnemonics

LDA, ADD, SUB, OUT, HIT represent the abbreviated form of the Instruction set, called Mnemonics.

5.3.2 Memory-Reference Instructions

LDA, ADD and SUB are memory referenced instructions because they use the data stored in memory. OUT and HLT are not memory referenced instructions since they do not use data stored in memory.

5.3.3 LDA

LDA stands for “Load the Accumulator” instruction.

A complete LDA instruction includes the hexadecimal address of the data to be loaded. LDA 8H for instance means load the Accumulator with the data 8H.

5.3.4 ADD

ADD is another SAP-1 instruction.

A complete ADD instruction includes the address of the word to be added.

For instance Add 9H means add the contents of memory location 9H to the contents of the accumulator; the sum replaces the original contents of accumulator. First the contents of 9H are loaded into B register, and instantly the Adder-Subtractor forms the sum of A and B.

5.3.5 SUB

A complete SUB instruction includes the address of the word to be subtracted.
For example SUB CH means subtract the contents of memory location CH from the contents of the accumulator.
First contents of CH are loaded into B register and then instantly Adder-Subtractor forms the difference of A and B.

5.3.6 OUT

The OUT instruction transfers the contents of accumulator to the Output port. After its execution the answer to the problem in the program can be seen on the LED display.
OUT is not memory referenced instruction; it does not need an address.

5.3.7 HLT

HLT stands for Halt. This instruction tells the computer to stop processing data.
HLT marks the end of a program similar to the way a period marks the end of a sentence.
You must use a HLT instruction at the end of every SAP-1 program; otherwise you get computer trash. HLT is complete in itself; it does not require RAM word since this instruction does not involve memory.

5.3.8 Programming SAP-1

To load an instruction in memory we have to use some kind of code that the computer can interpret. Following table shows this code. The number 0000 stands for LDA, 0001 for ADD, 0010 for SUB, 1110 for OUT and 1111 for HLT. Since this code tells the computer which operation to perform, it is called the Op-code.

LDA	0000
ADD	0001
SUB	0010
OUT	1110
HLT	1111

5.3.9 Instruction Decoder

A hex inverter produces complements of the opcode bits 17, 16, 15 and 14.
Then the 4 input NAND gates decode the five output signals LDA, ADD, SUB, OUT, HLT.
HLT is the only active low signal, while all others are active high.
When the HLT instruction is in the Instruction Register, all bits 17, 16, 15 and 14 are 1111 and HLT is low.
This signal returns to the single step clock (you made in Lab #1).
In either case that is AUTO or MANUAL the clock stops and the computer run ends.

5.3.10 Ring Counter

The Ring counter sometimes called the State Counter consists of three flip flop chips, 74LS107.

This counter is reset whenever the Clear-Start button S5 is pressed.
The output of the last flip flop is inverted so that the Q output drives the J input of the first flip flop. Due to this T1 output is initially high. The CLK signal drives an active low input; this means that the negative edge of the CLK signal initiates each T state. Output of Ring Counter is: $T = T_6T_5T_4T_3T_2T_1$.

At the beginning of a computer run, the ring word is T_1=000001
Successive clock pulses produce ring words:
T_2=000010
T_3=000100
T_4=001000
T_5=010000
T_6=100000

Then the Ring Counter reset to 000001 and the cycle repeats.
Each ring word represents one T state.
Fig 5.4 shows the timing pulses out of the Ring Counter.
The initial state T_1 starts with a negative clock edge and ends with next negative clock edge.
During this T state the T_1 output of the ring counter is high.
During the next state T_2 is high; the following state T_3 is high and so on.
As you can see the ring counter produces six T states.
Each instruction is fetched and executed during these six T states.

5.3.11 Address State

The T_1 state is called the address state because the address in the PC is transferred to the MAR during this state. During this state, EP and LM' are active; all other control bits are inactive. This means that the controller sequencer is sending out a control word of 12 bits shown as under:

	C_P	E_P	$\overline{L_M}$	$\overline{C_E}$	$\overline{L_I}$	$\overline{E_I}$	$\overline{L_A}$	E_A	S_U	E_U	$\overline{L_B}$	$\overline{L_O}$
CON	**0**	**1**	**0**	**1**	**1**	**1**	**1**	**0**	**0**	**0**	**1**	**1**

T_1 : MAR ← PC
T_2 : PC ← PC + 1
T_3 : IR ← M[MAR]

5.3.12 Increment State

During Increment State T_2 only C_P is active, causing the PC to increment to the next memory location.

CON	**1**	**0**	**1**	**1**	**1**	**1**	**1**	**0**	**0**	**0**	**1**	**1**

5.3.13 Memory State

The T_3 state is called the memory state because the addressed RAM instruction is transferred from the memory to the Instruction register. The only active control bits are C_E' and L_I' and word out of the controller is

CON	**0**	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**0**	**1**	**1**

5.3.14 Fetch cycle

The address, increment and memory states are called the fetch cycle of SAP-1

5.3.15 Execution cycle

The next three states T_4, T_5 and T_6 are the three states of the execution cycle of SAP-1. The register transfer during this execution depends on a particular instruction being executed. Each instruction has its own control routine.

1. The LDA, SUB, ADD and OUT signals from the instruction decoder drives the Control Matrix.
2. At the same time, the Ring Counter signals, T_1 to T_6 also drive the matrix.
3. The matrix produces CON, a 12-bit microinstruction that, tells the rest of the computer what to do.

   State CON Active Bits

   T1 5E3H E_P, L_M'
   T2 BE3H Cp
   T3 263H C_E', L_I'

4. The Control unit is the key to a computer's automatic operation.
5. The Control unit generates the control words that fetch and execute each instruction.
6. While each instruction is fetched and executed, the computer passes through different T states, or timing states, that is periods during which register contents change.

5.4 Procedure

1. Connect the circuit as shown in the circuit diagram.
2. Connect 17, 16, 15 and 14 to dip switches on your trainer and give the different op-codes for different instructions.
3. For instance for HLT' instruction, op-code is 1111, this means keeping all dip switches in "Hi" position will generate an active low HLT' signal, similarly you can check for other instructions.
4. Check the signals with the oscilloscope and get your work verified by your Lab instructor.
5. You will be graded for this Lab on the output and functionality of your circuit.

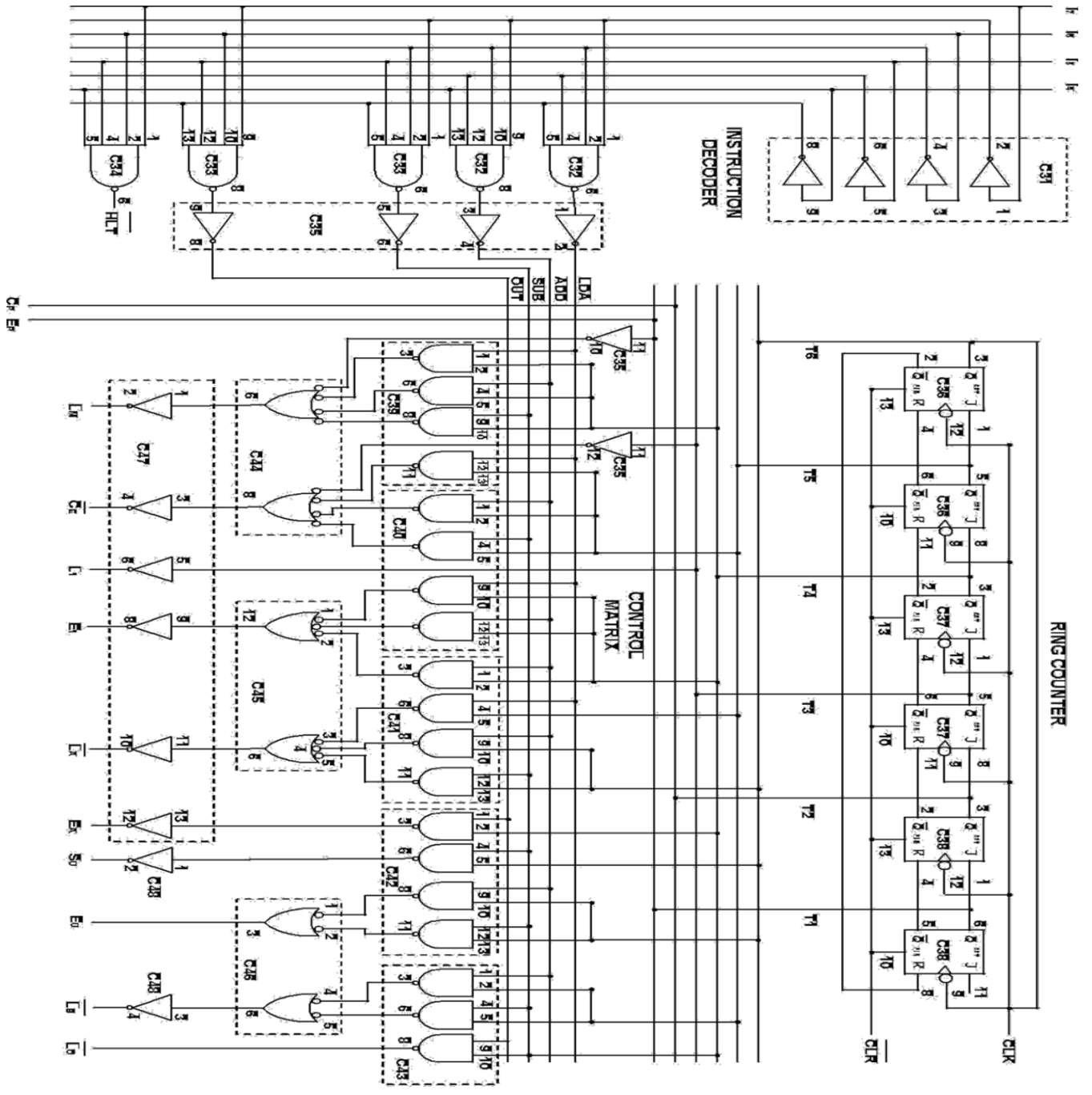
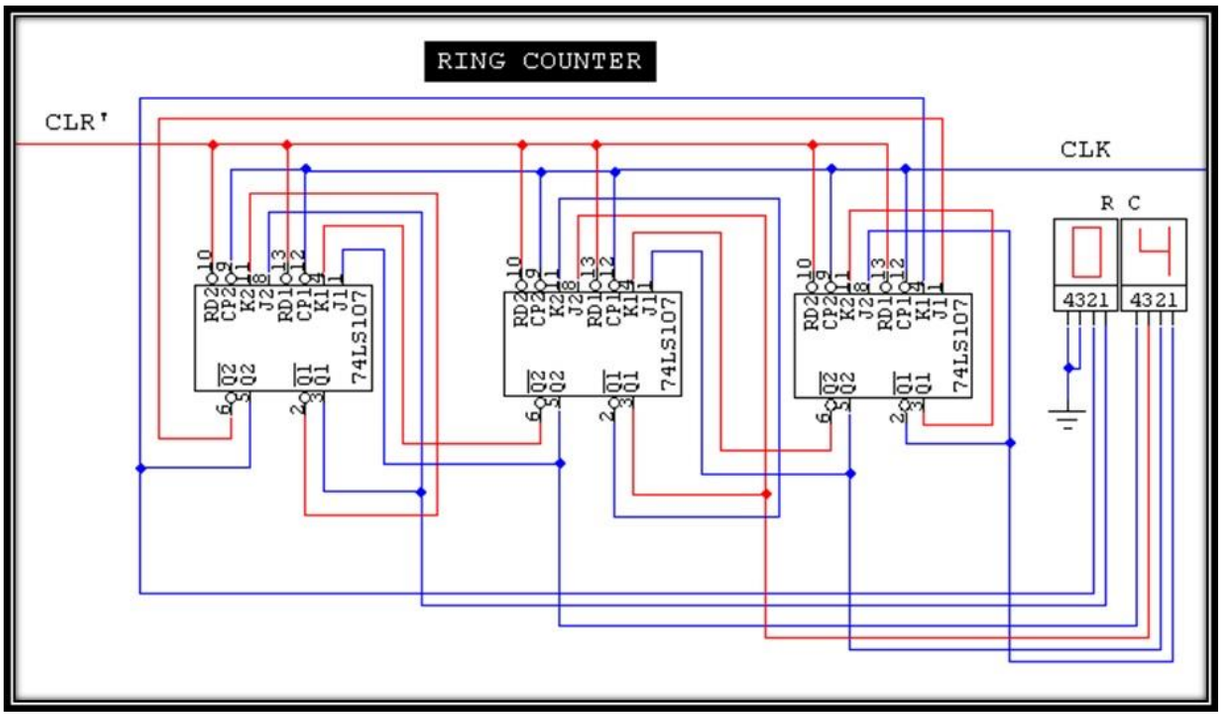
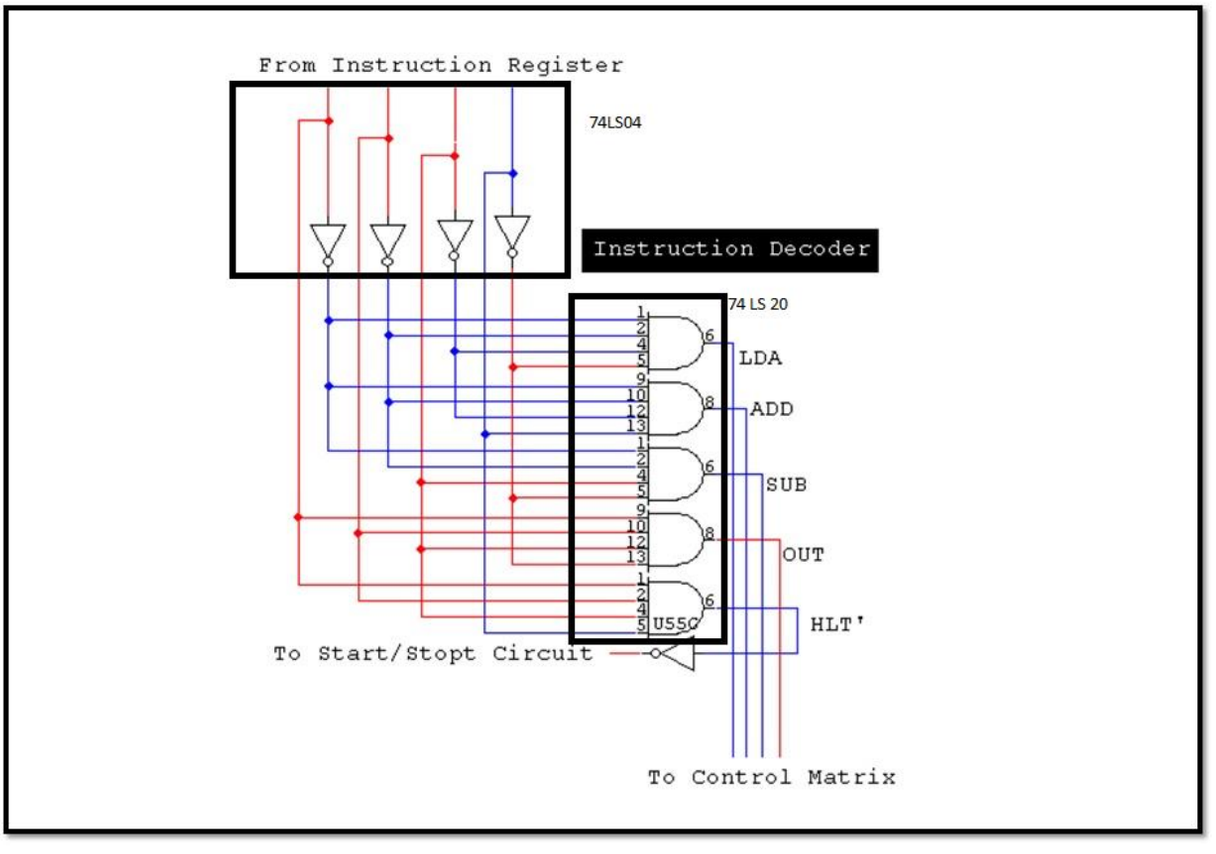


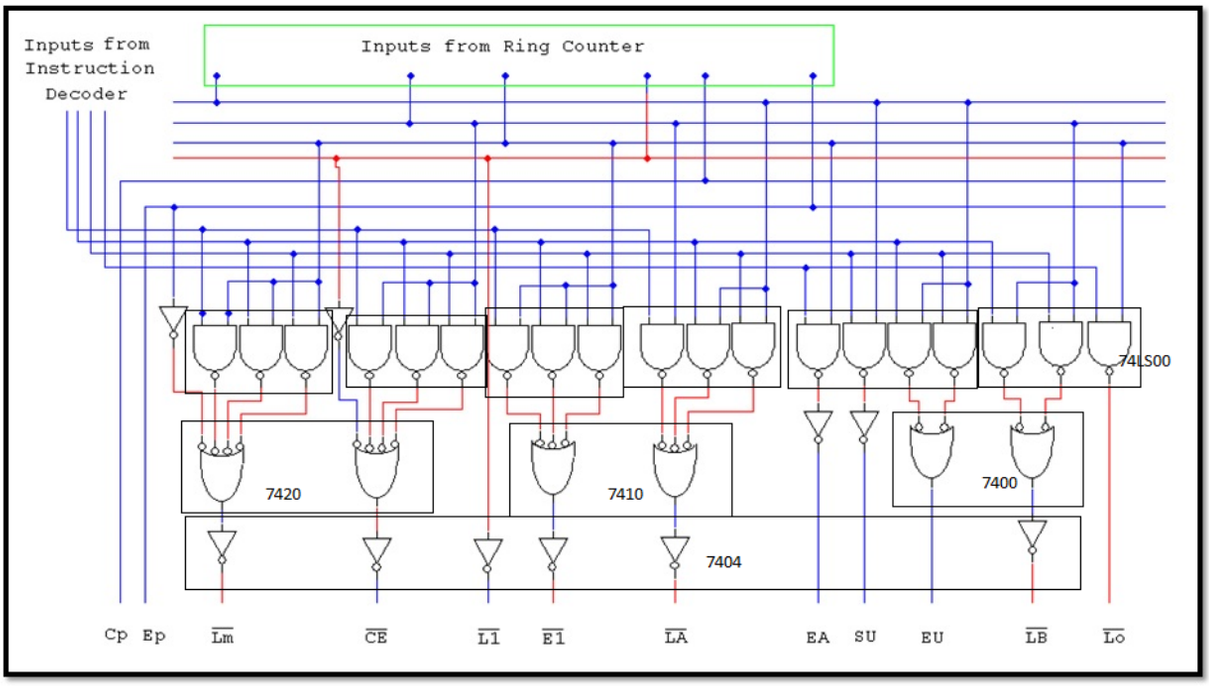
Figure 5-1: Instruction Decoder, Ring Counter and Control Matrix

Separate diagrams for ring counter instruction decoder and control matrix are as follows for better visibility





CONTROL MATRIX



Lab – 6: Accumulator & Adder/Subtractor

6.1 Objective

To make the circuit of accumulator register and a 4-bit adder/Subtractor.

6.2 Components Required

a) 4 bit D-type Register IC, 74LS173 … 2
b) 4 bit Bus Buffer (Quad three state switch) IC, 74LS126 … 4
c) 4 bit Full Adder IC, 74LS83 … 2
d) Connecting wires

6.3 Accumulator

The Accumulator is a buffer register that stores intermediate answers during a program execution. The accumulator has both a two state and a three state output. The two state output goes directly to the adder/Subtractor and the three state output goes to the W bus via a buffer. Therefore, the 8 bit Accumulator word continuously drives the adder/Subtractor; the same word appears on the W bus when E_A is high. Chips C10 and C11, 74LS173s, make up the accumulator. Pins 1 and 2 are grounded on both chips to produce a two state output for the adder/Subtractor. Chips C12 and C13 are 74LS126s; these three-state switches place the accumulator contents on the W bus when E_A is high.

6.4 Adder/Subtractor

SAP-1 uses a 2's complement Adder/Subtractor.

When SU is low, the sum out of the Adder-Subtractoris A = A + B.
When SU is high, the difference appears A = A + B' + 1 (Recall that the 2's complement is equivalent to a decimal sign change).
The adder/Subtractor is asynchronous (un-clocked); this means that its contents changes as soon as the input words change. When E_U is high, these contents appear on the W bus Chips C14 and C15 are 74LS86s. These ex-or gates are a controlled inverter. When SU is low, the contents of the B register are transmitted. When Su is high, the 1's complement is transmitted and a 1 is added to the LSB to form the 2's complement. Chips C16 and C17 are 74LS83s. These 4-bit full adders combine to produce an 8-bit sum or difference. Chips C18 and C19, which are 74LS128s, convert the 8-bit answer into a three-state output for driving the W bus.

6.5 Procedure

Make the connections as shown in circuit diagram.

6.6 What shall be the Lab's Outcome?

The input to the Accumulator should be through switches, which will be one 8-Bit data number to add/sub. There should be a static 8- bit number (let's say binary one) as the other input to the adder/Subtractor. The output of the adder/Subtractor should be connected to LEDs. By proper input to the S_U pin. The LEDS should be showing the results of adder/Subtractor.

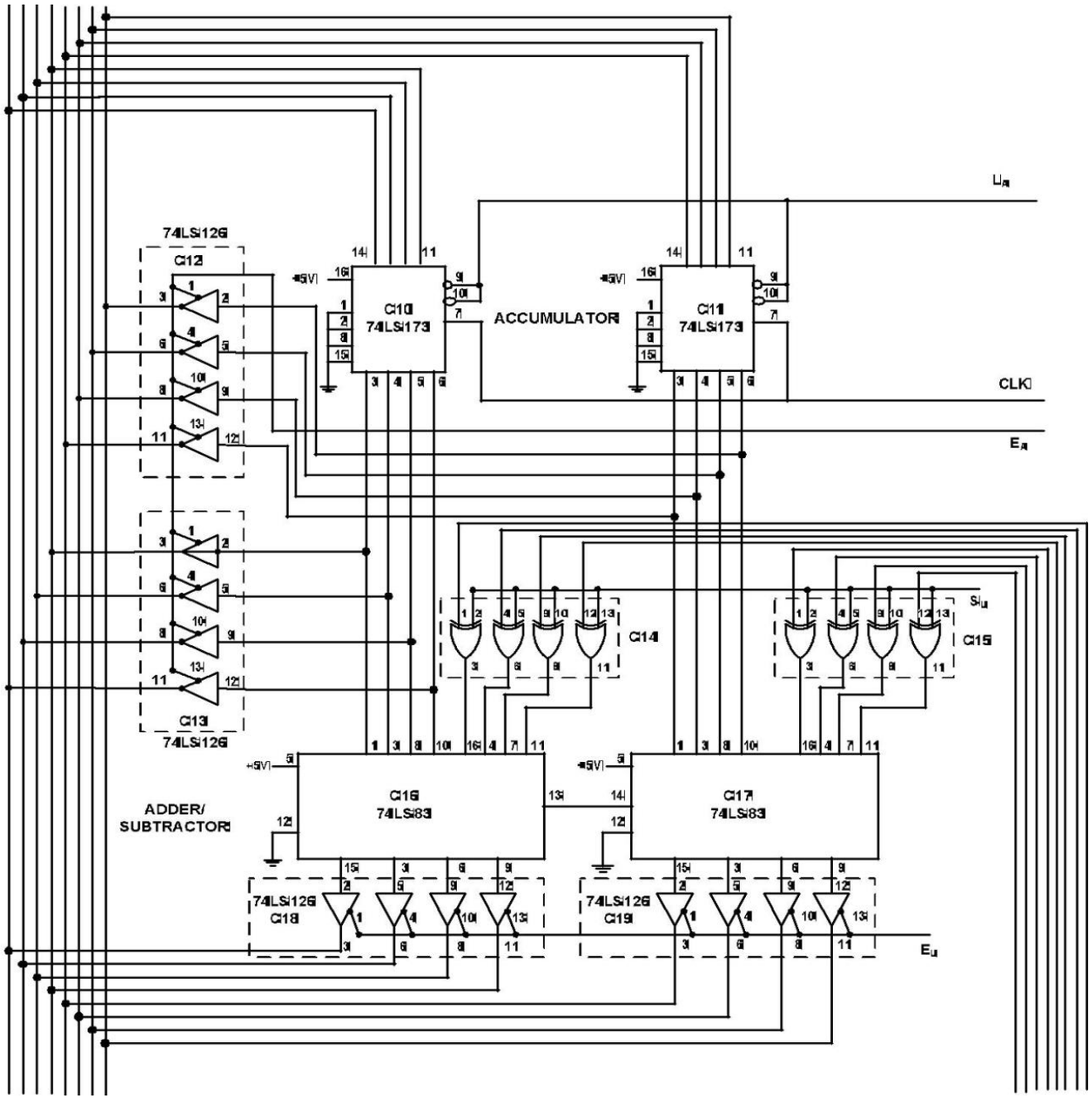
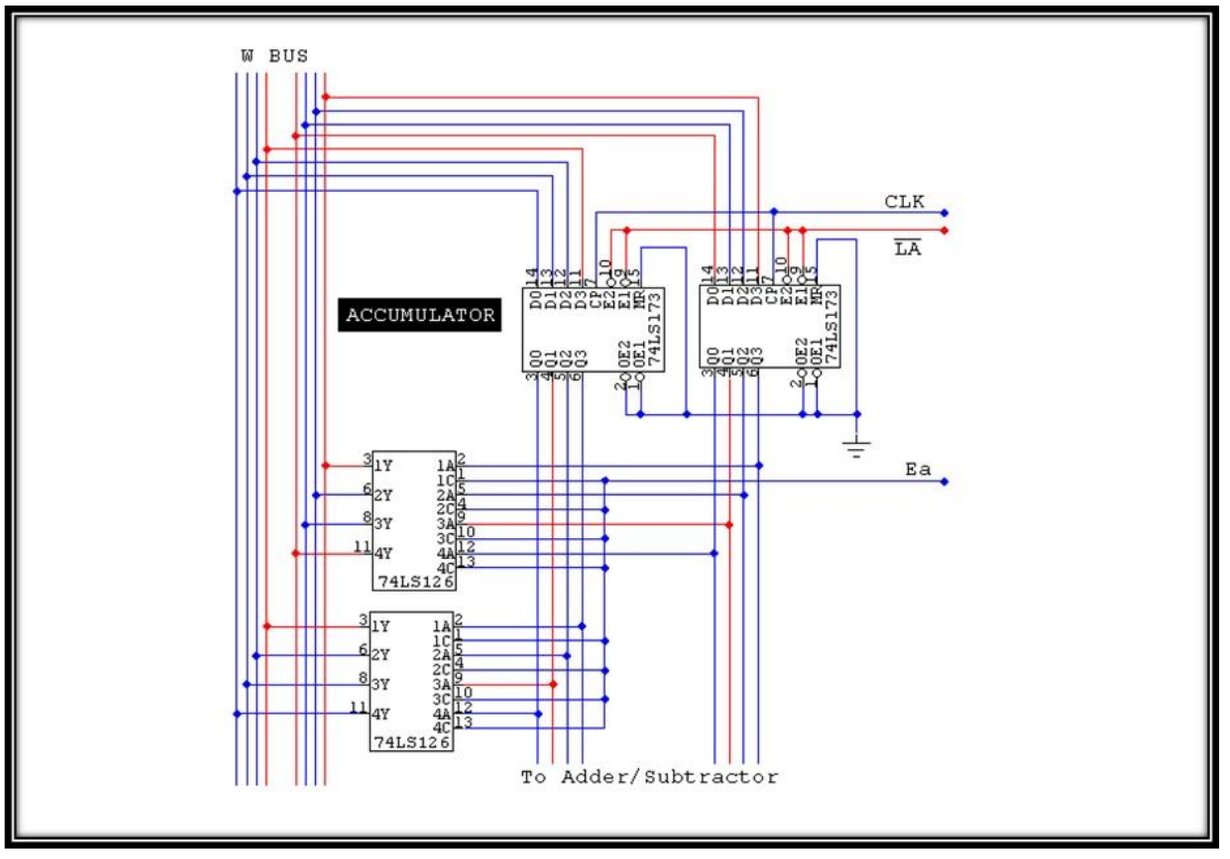
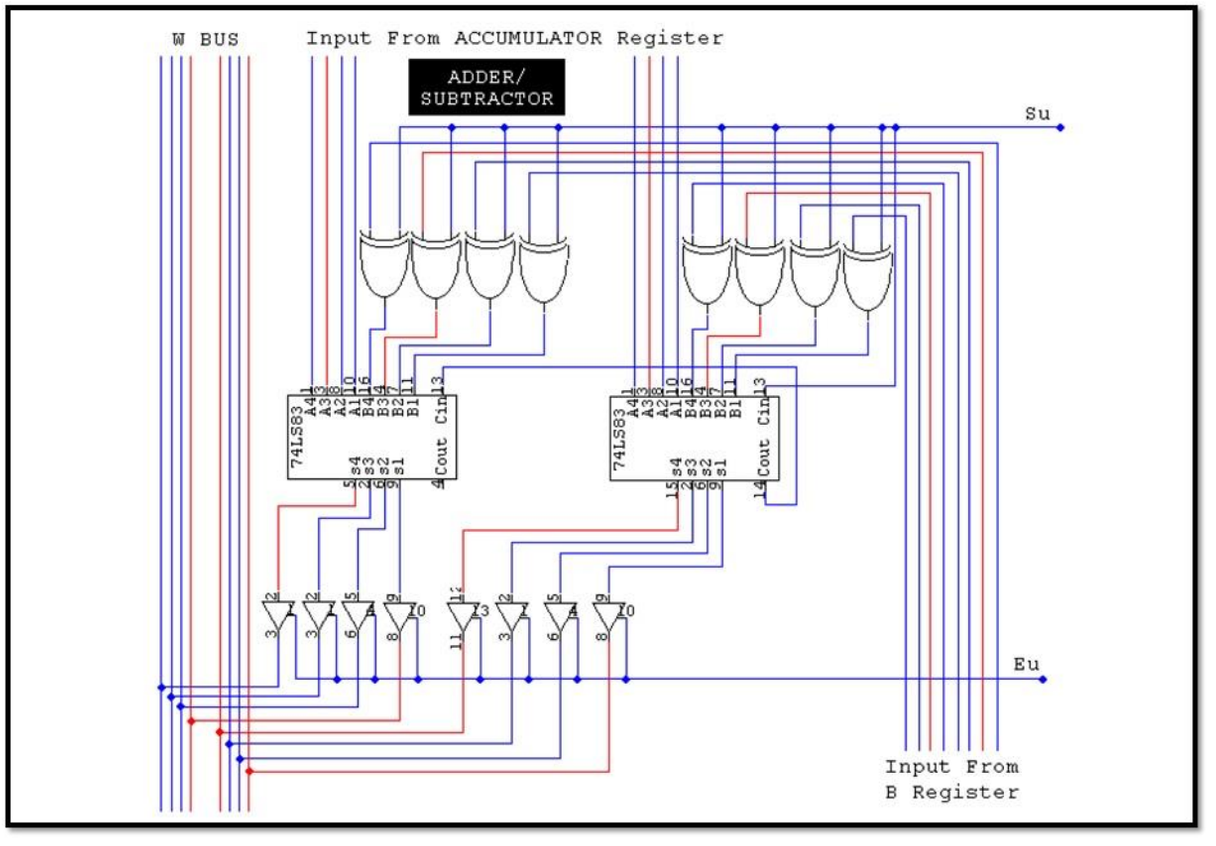
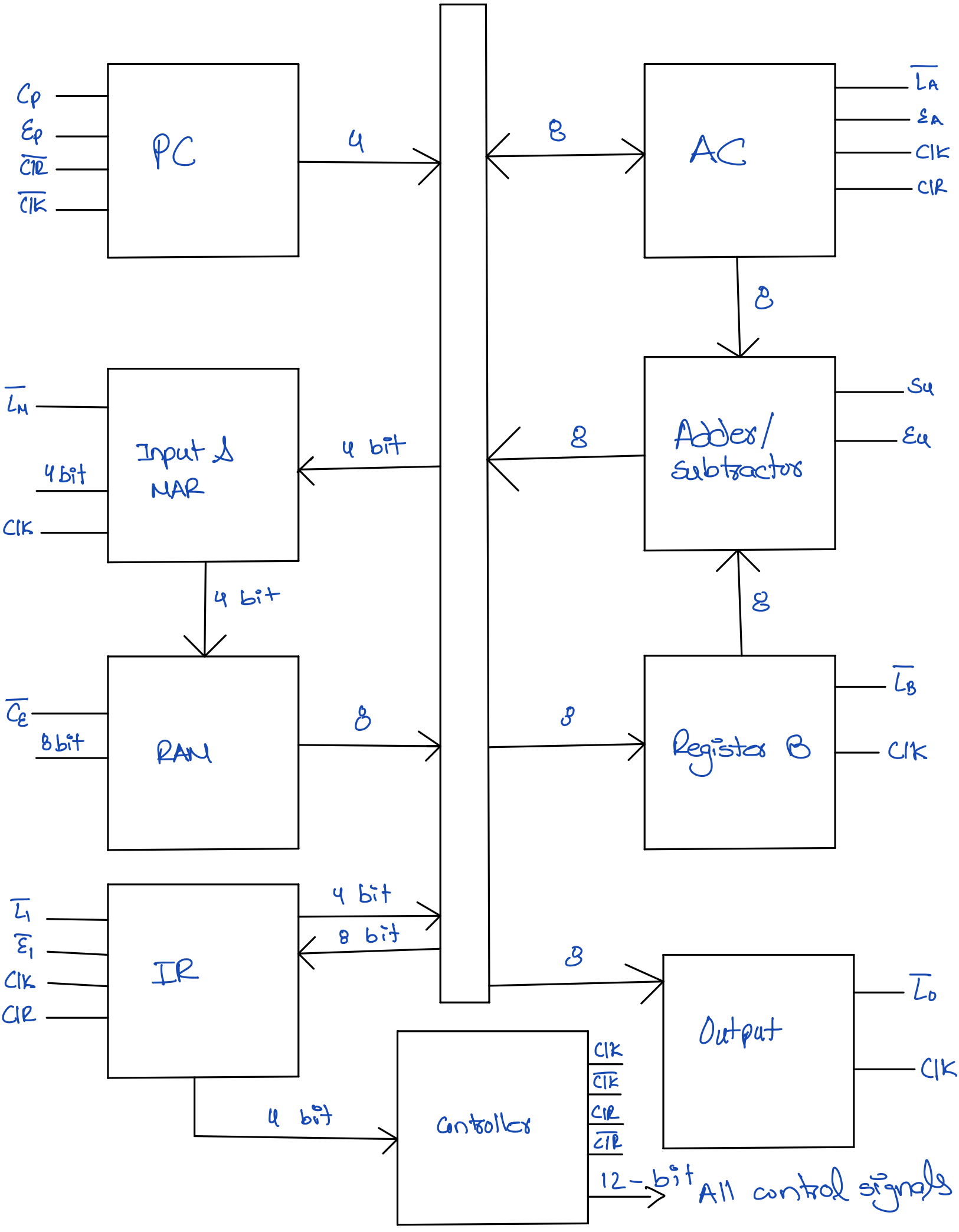


Figure 6-1: Accumulator and Adder/Subtractor


W BUS
CLK
LA
ACCUMULATOR
74LS173
74LS173
Ea
74LS126
74LS126
To Adder/Subtractor


W BUS
Input From ACCUMULATOR Register
ADDER/
SUBTRACTOR
Su
74LS83
74LS83
Eu
Input From
B Register


PC
C_P
E_P
$\overline{CIR}$
$\overline{CIK}$
4
8
AC
$\overline{L_A}$
E_A
CIK
CIR
8
Input & MAR
$\overline{L_M}$
4 bit
CIK
4 bit
4 bit
Adder/ Subtractor
8
S_4
E_4
8
RAM
$\overline{C_E}$
8 bit
8
8
Register B
$\overline{L_B}$
CIK
IR
$\overline{L_I}$
$\overline{E_I}$
CIK
CIR
4 bit
8 bit
8
Output
$\overline{L_O}$
CIK
4 bit
Controller
CIK
$\overline{CIK}$
CIR
$\overline{CIR}$
12-bit
All control signals

Lab – 7: B-Register, Output Register and Binary Display

7.1 Objective

To make and connect B-register and output-register through the Bus, And to read the contents of output register through binary display.

7.2 Components Required

a) 4 bit D type register IC, 74LS1734 … 4
b) LEDs … 8
c) Connecting Wires

7.3 B Register

The B register is a buffer register. It is used in arithmetic operations. A low L_B and a positive clock edge load the word on the W bus into the B register. The two state output of the B register drives the Adder/ Subtracter, supplying the number to be added or subtracted from the contents of the accumulator.

7.4 Output Register

At the end of the computer run the accumulator contains the answer to the problem being solved, at this point, we need to transfer the answer to the outside world, and this is where the output register is used. When EA is high and Lo is low, the next positive clock edge loads the accumulator word into the output register.

7.5 Binary Display

The binary display is a row of eight light-emitting LEDs because each LED is connected to one pin of the output register the binary display shows the contents of the output register.

7.6 Procedure

Connect the circuit as shown in the circuit diagram.

7.7 What shall be the lab's outcome?

At the end of today's lab there should be perfect coordination between accumulator, register, adder /Subtractor, B register, output register and Binary Display

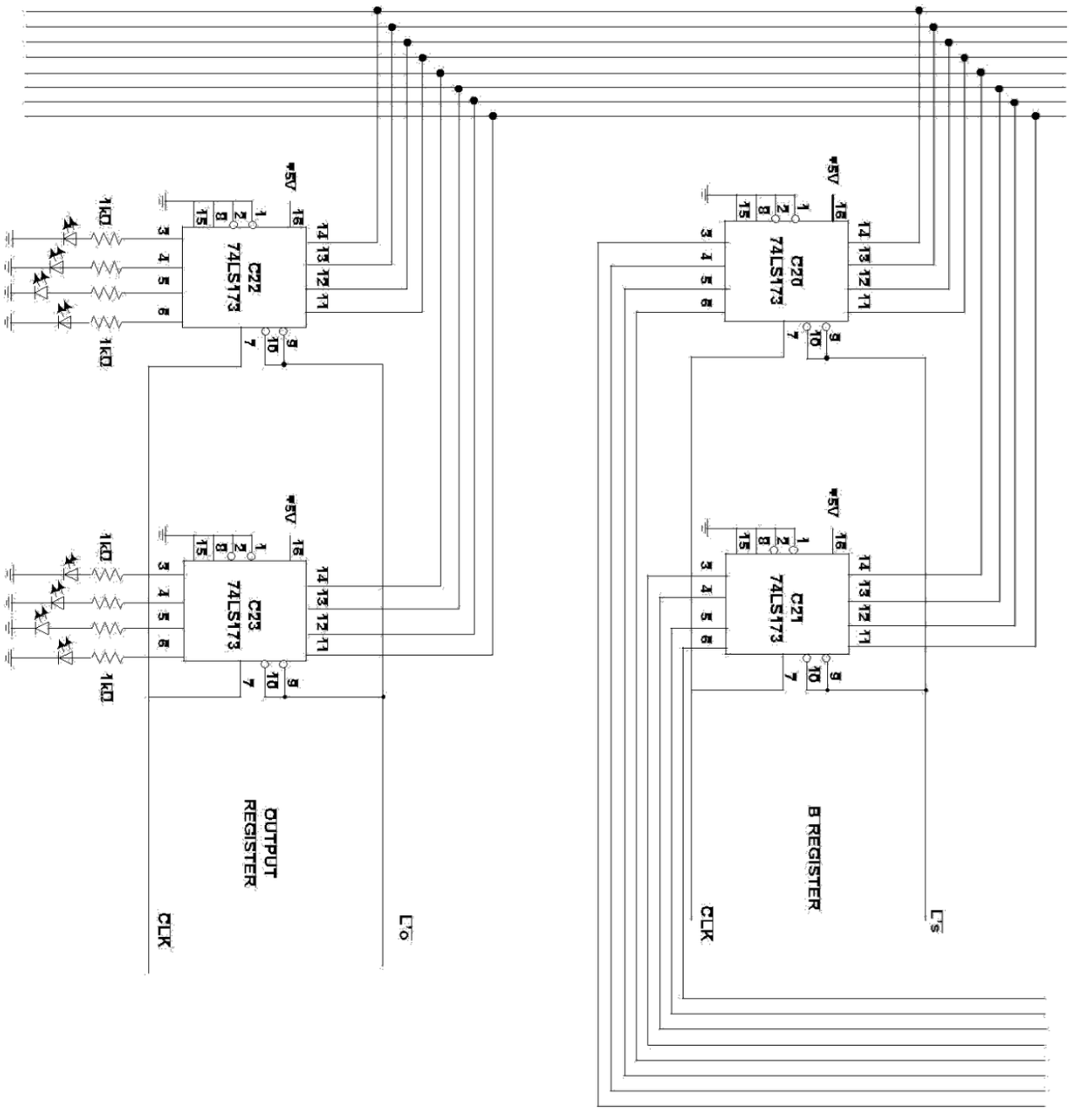
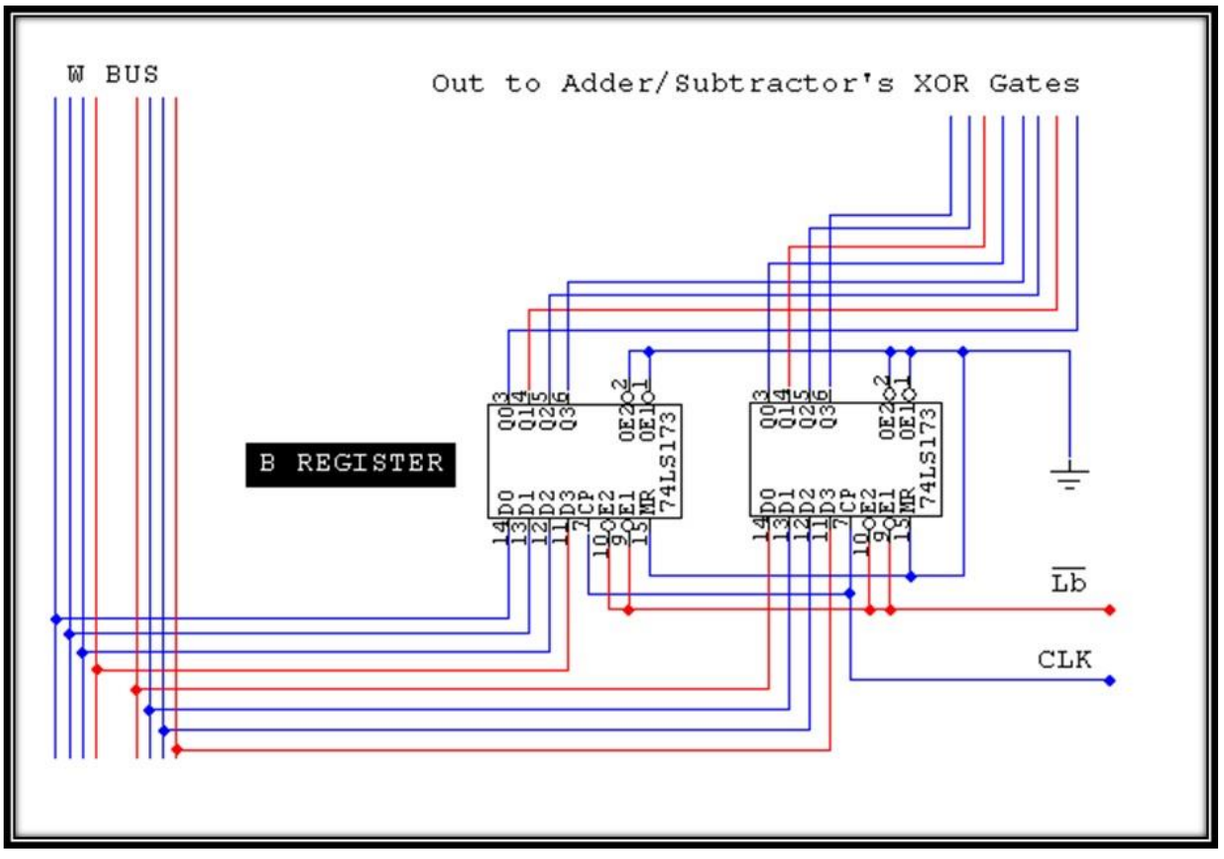
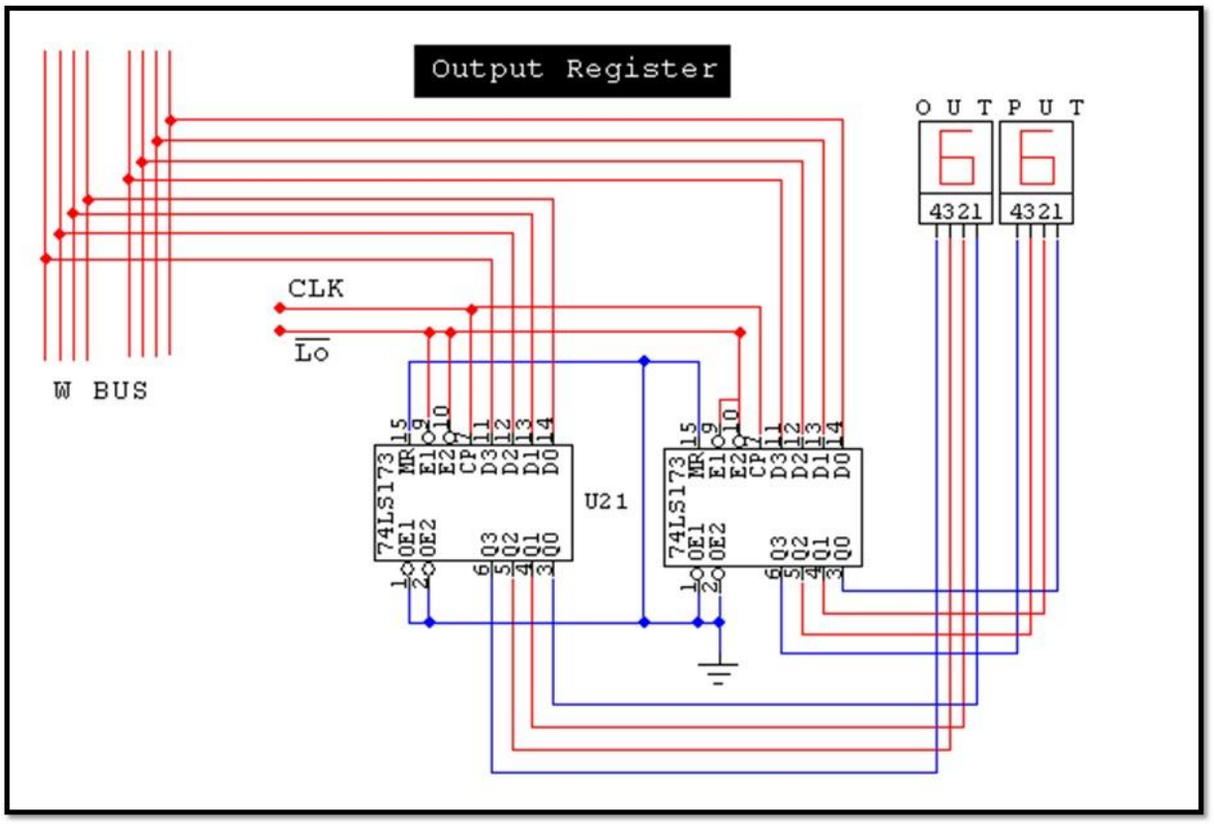


Figure 7-1: B-Register, Output Register, and Binary Display


W BUS
Out to Adder/Subtractor's XOR Gates
B REGISTER
74LS173
74LS173
Lb
CLK


Output Register
O U T P U T
4321
4321
CLK
Lo
W BUS
74LS173
74LS173
U21

Lab – 8: Checking of SAP-I and Introduction to HDL Based Designing

8.1 Objectives

In this lab connect all the blocks of Sap-I, check each circuit and run the SAP-I completely. (Given as a hardware project). This lab covers the Complex Engineering Problem (CEP) part of the course.

8.2 Introduction

HDL (Hardware Description Language) is any language from a class of computer languages, specification languages, or modeling languages for formal description and design of electronic circuits, and most-commonly, digital logic. It can describe the circuit's operation, its design and organization, and tests to verify its operation by means of simulation. The two most popular HDLs are Verilog and VHDL. Verilog due to its similarity to C language is easier to understand so has become most widely used HDL in educational institutions.

8.2.1 Design Styles

There are two basic types of digital design methodologies: a top-down design methodology and a bottom-up design methodology.

In a **top-down design methodology**, we define the top-level block and identify the sub- blocks necessary to build the top-level block. We further subdivide the sub-blocks until we come to leaf cells, which are the cells that cannot further be divided.

In a **bottom-up design methodology**, we first identify the building blocks that are available to us. We build bigger cells, using these building blocks. These cells are then used for higher-level blocks until we build the top-level block in the design.

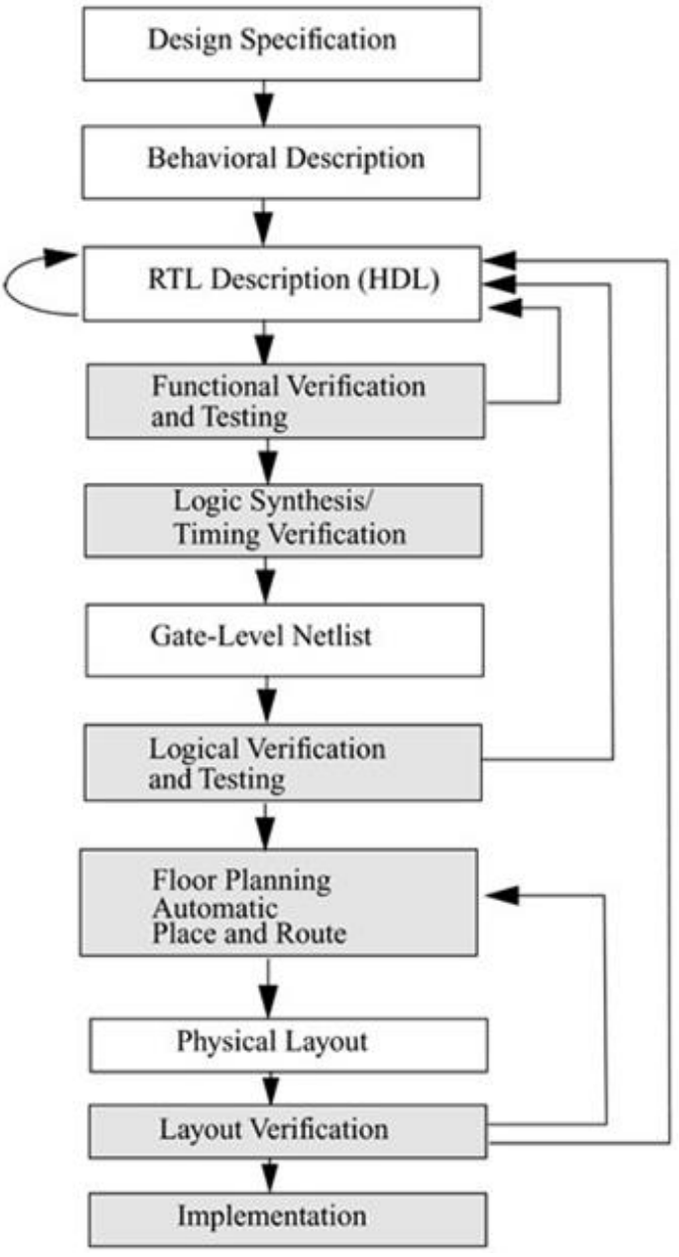


Figure 8.1 Design Styles

8.2.2 Module

Verilog provides the concept of a module. A module is the basic building block in Verilog. A module can be an element or a collection of lower-level design blocks.

In Verilog, a module is declared by the keyword **module**. A corresponding keyword **endmodule** must appear at the end of the module definition. Each module must have a *module_name*, which is the identifier for the module, and a *module_terminal_list*, which describes the input and output terminals of the module.

module <*module_name***> (<***module_terminal_list***>); //outputs first then inputs**
...
<*module internals*> // **Also called module body**
...
endmodule

8.2.3 Lexical Conventions

The basic lexical conventions used by Verilog HDL are similar to those in the C programming language. Verilog contains a stream of tokens. Tokens can be comments, delimiters, numbers, strings, identifiers, and keywords. Verilog HDL is a case-sensitive language. All keywords are in lowercase.

8.2.4 Whitespace

Blank spaces (\b), tabs (\t) and newlines (\n) comprise the whitespace. Whitespace is ignored by Verilog except when it separates tokens. Whitespace is not ignored in strings.

8.2.5 Comments

Comments can be inserted in the code for readability and documentation. There are two ways to write comments. A one-line comment starts with "//". Verilog skips from that point to the end of line. A multiple-line comment starts with "/*" and ends with "*/". Multiple-line comments cannot be nested. However, one-line comments can be embedded in multiple-line comments.

```
a = b && c; // This is a one-line comment

/* This is a multiple line comment */
```

8.2.6 Operators

Operators are of three types: unary, binary, and ternary. Unary operators precede the operand. Binary operators appear between two operands. Ternary operators have two separate operators that separate three operands.

```
a = ~ b; // ~ is a unary operator. b is the operand
a = b && c; // && is a binary operator. b and c are operands
a = b ? c : d; // ?: is a ternary operator. b, c and d are operands
```

8.2.7 Number Specification

There are two types of number specification in Verilog: sized and unsized.

a) Sized numbers

Sized numbers are represented as *<size> '<base format> <number>.*
<size> is written only in decimal and specifies the number of bits in the number. Legal base formats are decimal ('d or 'D), hexadecimal ('h or 'H), binary ('b or 'B) and octal ('o or 'O). The number is specified as consecutive digits from 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f.

Only a subset of these digits is legal for a particular base. Uppercase letters are legal for number specification.

4'b1111 // this is a 4-bit binary number
12'habc // this is a 12-bit hexadecimal number
16'd255 // this is a 16-bit decimal number.

b) Un-Sized Numbers

Numbers that are specified without a <base format> specification are decimal numbers by default. Numbers that are written without a <size> specification have a default number of bits that is simulator- and machine-specific (must be at least 32).

23456 // this is a 32-bit decimal number by default
'hc3 // this is a 32-bit hexadecimal number
'o21 // this is a 32-bit octal number

8.2.7 Strings

A string is a sequence of characters that are enclosed by double quotes. The restriction on a string is that it must be contained on a single line, that is, without a carriage return. It cannot be on multiple lines. Strings are treated as a sequence of one-byte ASCII values.

"Hello Verilog World" // is a string "a / b" // is a string

8.2.8 Identifiers and Keywords

Keywords are special identifiers reserved to define the language constructs. Keywords are in lowercase. Identifiers are names given to objects so that they can be referenced in the design.
Identifiers are made up of alphanumeric characters, the underscore (_), or the dollar sign ($). Identifiers are case sensitive. Identifiers start with an alphabetic character or an underscore.
They cannot start with a digit or a $ sign (The $ sign as the first character is reserved for system tasks).

reg value; // reg is a **keyword**; value is an identifier input clk; // input is a keyword, clk is an identifier

8.2.9 Simulation

Once a design block is completed, it must be tested. The functionality of the design block can be tested by applying stimulus and checking results. We call such a block the **stimulus block**. It is good practice to keep the stimulus and design blocks separate. The stimulus block can be written in Verilog. A separate language is not required to describe stimulus. **The stimulus block is also commonly called a test bench**. Different test benches can be used to thoroughly test the design block.
Stimulus block is usually of the form:

module Stimulus;
reg <inputs> //all inputs become reg

wire <outputs> //all outputs becomes wire
<main_module instance> //Instantiation will be discussed in more detail in future labs
<Simulation conditions with timings> //which give specific outputs related to design
endmodule

Lab – 9: Introduction to Xilinux IDE

9.1 Introduction

Xilinx ISE (**I**ntegrated **S**ynthesis **E**nvironment) is a software tool produced by Xilinx for the synthesis and analysis of HDL designs, enabling the developer to synthesize ("compile") their designs, perform timing analysis, examine RTL diagrams, simulate a design's reaction to different stimuli, and configure the target device with the programmer.

Xilinx ISE is a design environment for FPGA products from Xilinx, and is tightly-coupled to the architecture of such chips, and cannot be used with FPGA products from other vendors. The Xilinx ISE is primarily used for circuit synthesis and design, while ISIM or the ModelSim logic simulator is used for system-level testing. Other components shipped with the Xilinx ISE include the Embedded Development Kit (EDK), a Software Development Kit (SDK) and ChipScope Pro.

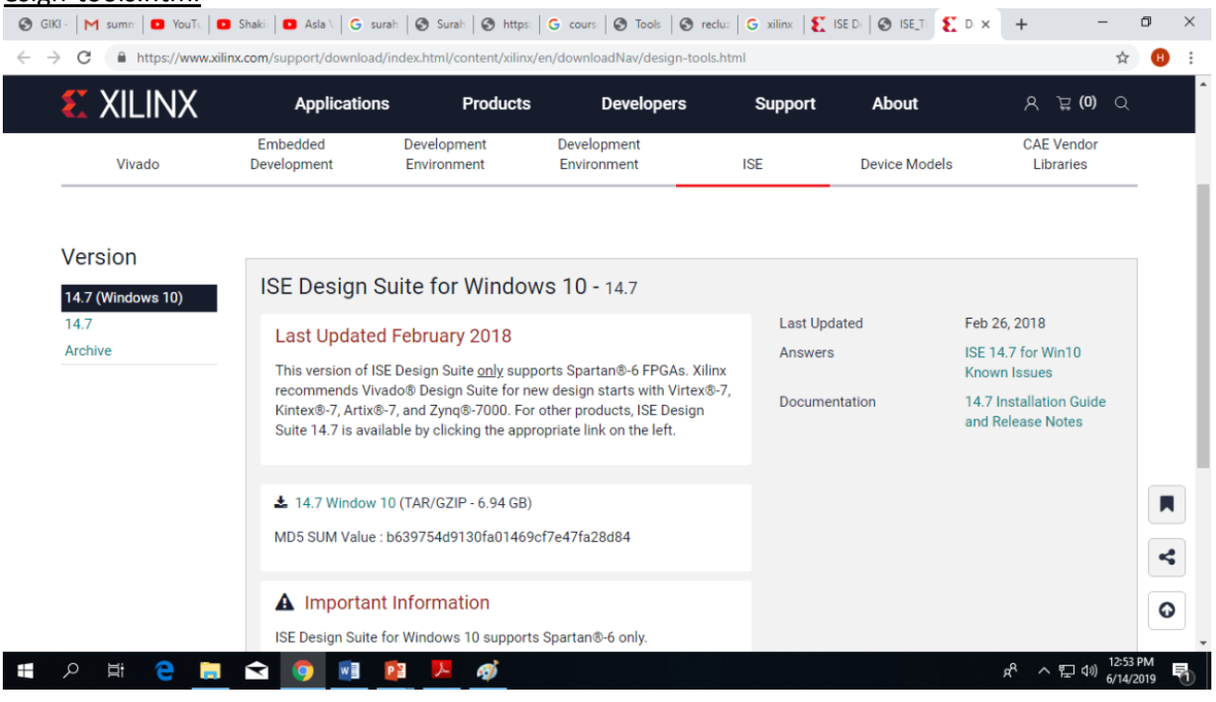
9.2 Installation Guide

The latest version of the software can be downloaded freely from the following site based on the specification of the system being used especially for WiN10 users.

https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/design-tools.html

Installation Notes for Xilinx 14.7 on Windows 10

1. These instructions are for installing the Xilinx ISE version 14.7 on Windows 10 only. For using the tool on Windows 10, there are two options: Go to the following link, https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/design-tools.html

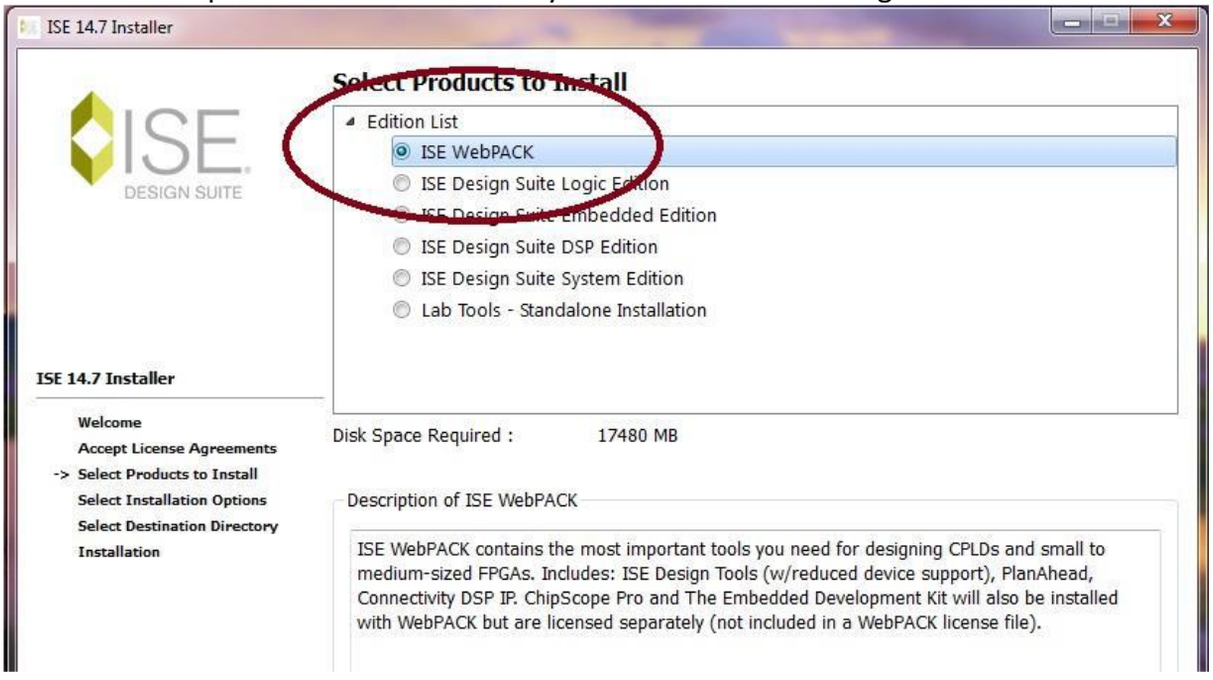


Select 14.7(Windows 10) which supports Windows 10 Pro or Windows 10 Enterprise.

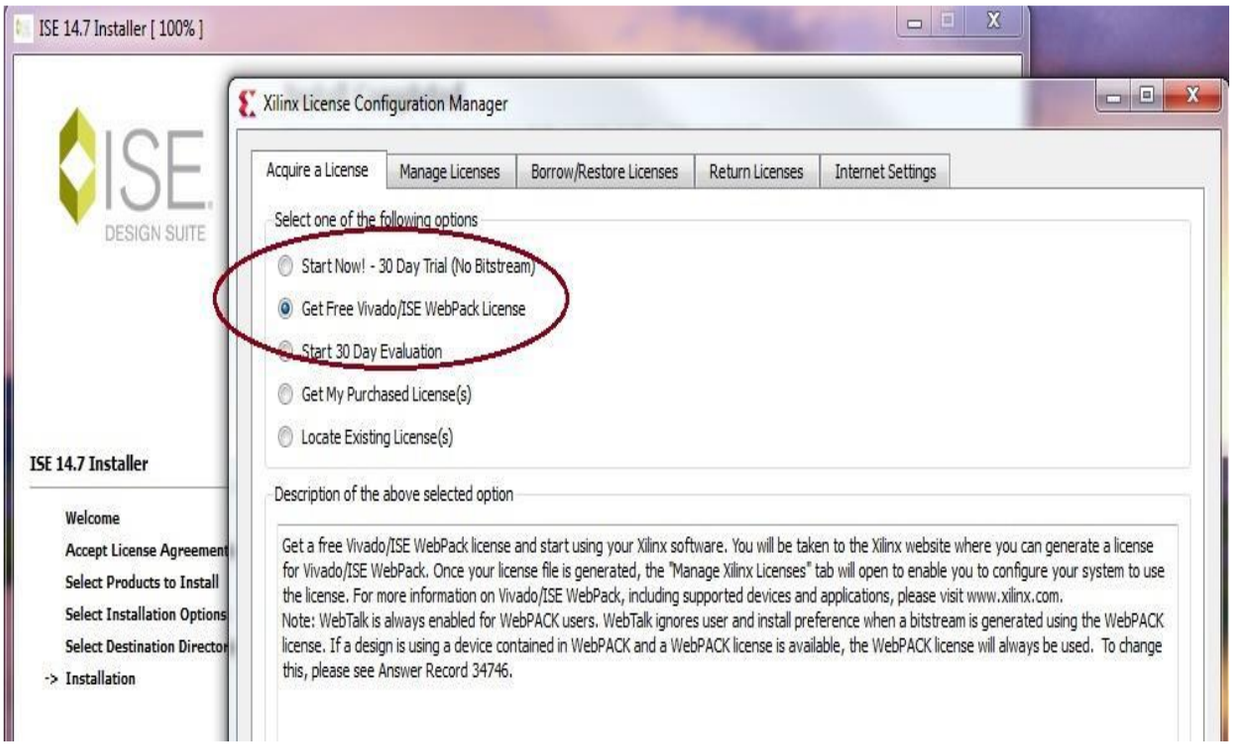
Or Go to the Xilinx website and to the **Downloads** option under **Support** and select the **ISE Design Tools** tab, or follow the link below:

http://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/design-tools.html

2. Go to the extracted folder and run **xsetup**, it will take a few moments for it to pop open, it is much better if you are not doing this over a network and your anti-virus is disabled,
3. After clicking **Next** on the initial page and accepting the terms and conditions (two pages) you need to select which package to install. You should download the first option **ISE WebPack** as it is the version that works with the free license, the other versions require a full license and they are about three GBs larger.



4. Do not change anything in the next page that is **Installation Options**, and leave all options checked. I recommend not changing the program path but do so if you want it installed in another specific place. Follow the summary page and click **install** when you get to it.
5. It takes around 30 minutes to an hour for installation , but keep in mind that you will be prompted a couple of times when it gets to about 90% of the installation.
6. When the installation is complete the **License Configuration Manager** will automatically pop up. Make sure to choose the second option **Get Free Vivado/ISE webpack License**, do not get the 30 day option as it will not be appropriate for our purposes. This program will not work without this free **Webpack** license.

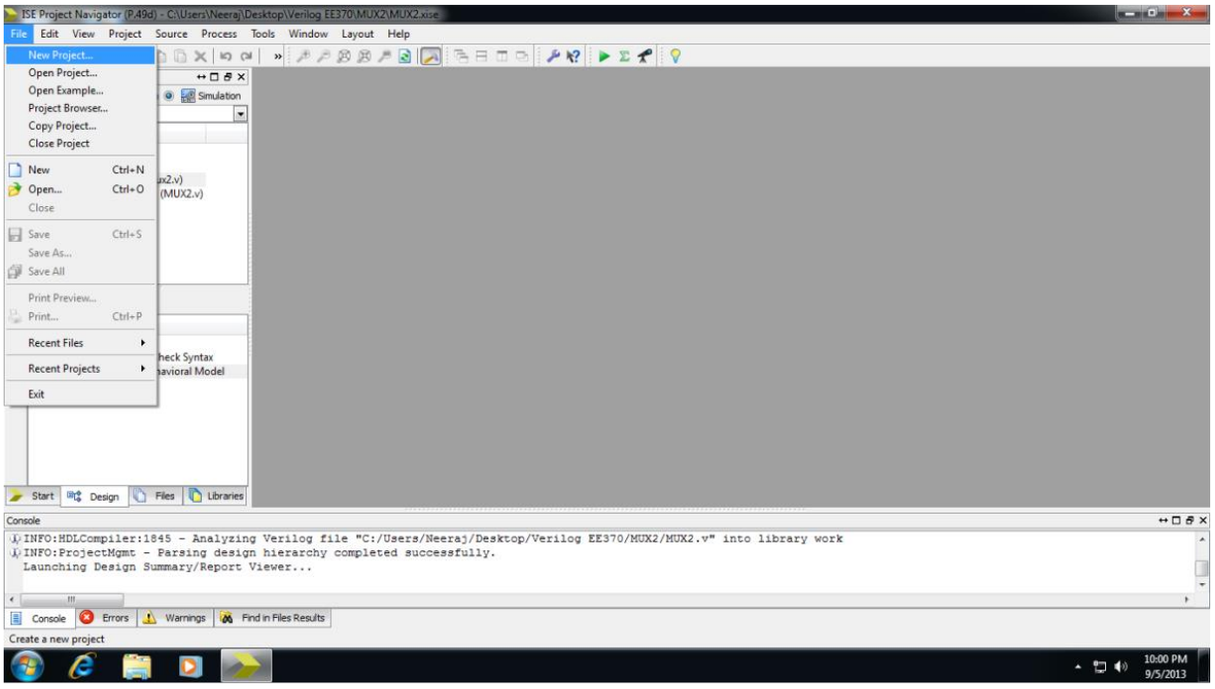


9.3 Starting a New Project

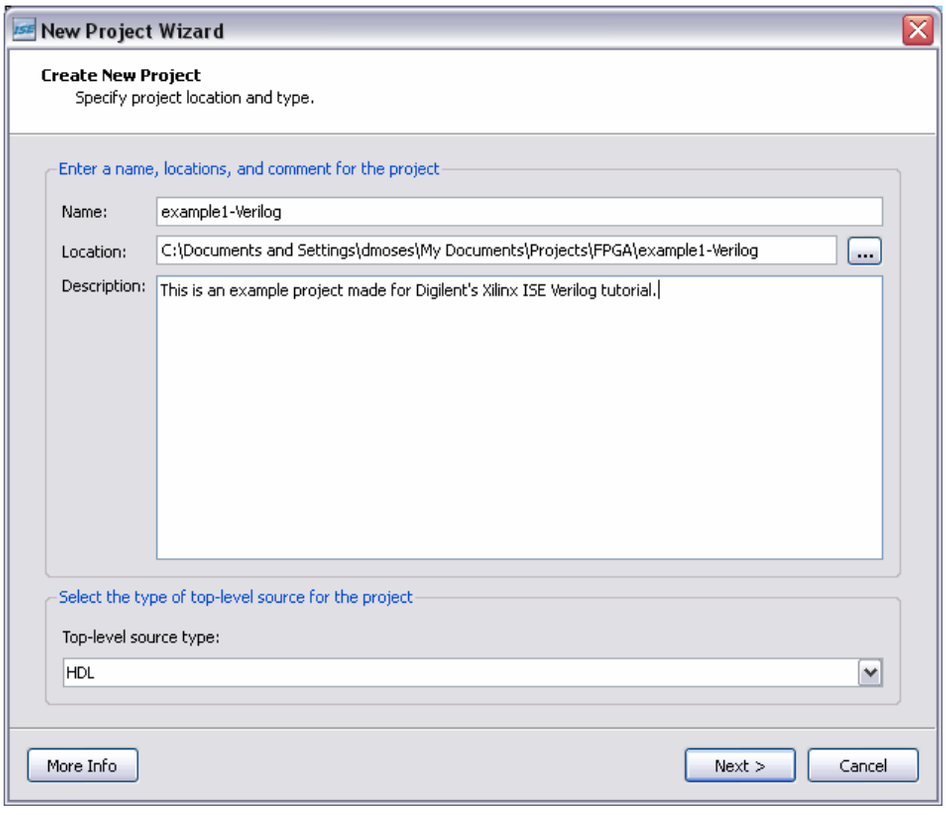
1. To create a new project, open Project Navigator either from the Desktop icon or by selecting

    a. Start > Programs > Xilinx ISE Design Suite 11 > ISE > Project Navigator. In Project Navigator,
    b. Select the New Project option from the Getting Started menu (or by selecting Select File > New Project).
    c. This brings up a Dialog box where you can enter the desired project name and project location.
    d. Choose a meaningful name for easy reference. Here we call this project “example1-Verilog” and save it in a local directory. You can place comments for your project in the Description text box. We use HDL for our top-level source type in this tutorial

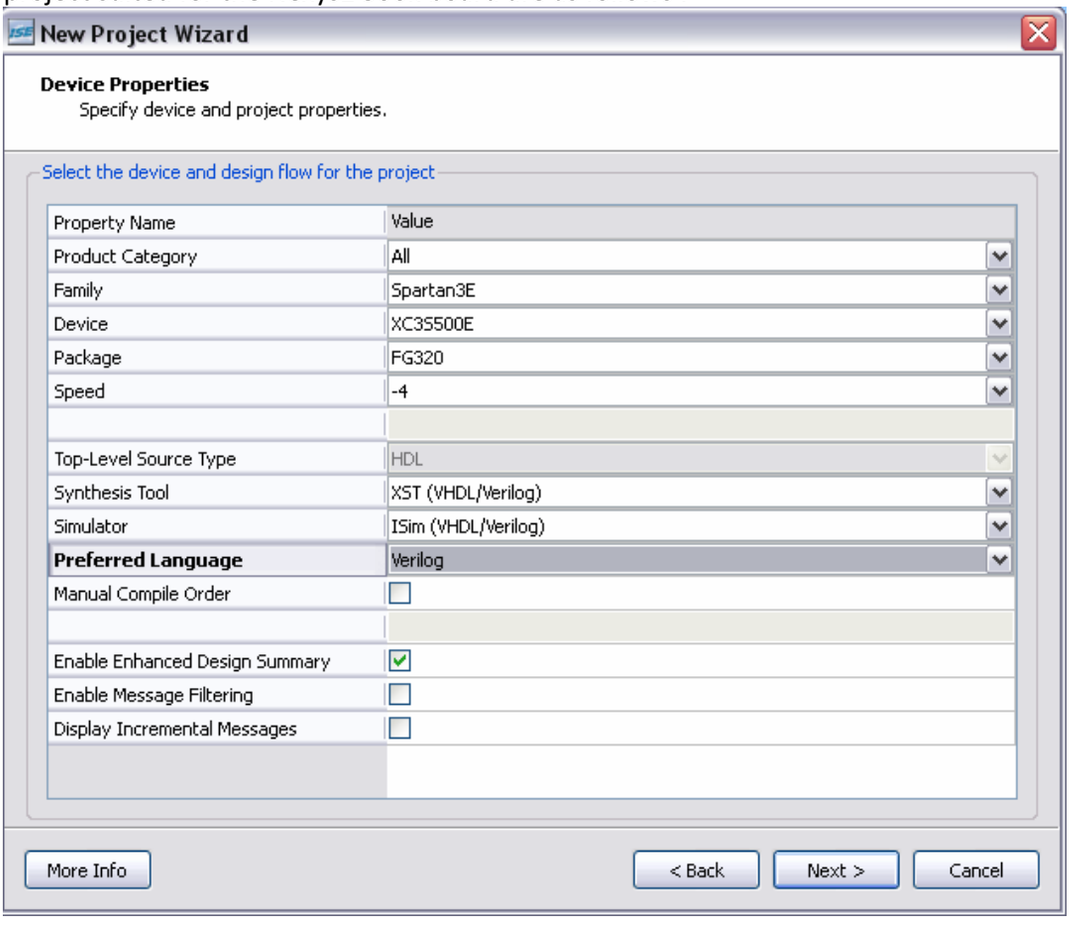
9.3.1 Open New Project



Choose the location to create New Project. Always choose something other than User folder as it is read only. The best way is to create a folder on desktop, name it and copy its *url* and paste it in the "location" of the "new project wizard".

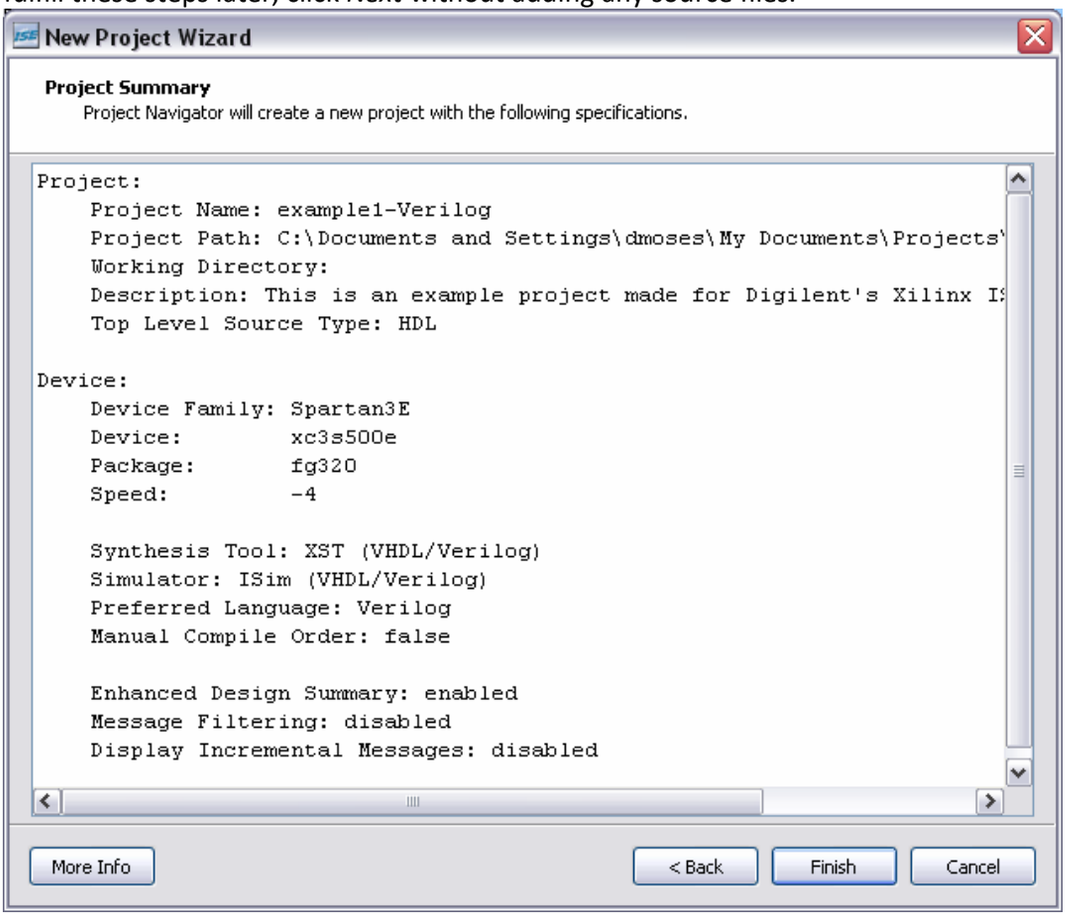


2. (The next step is to select the proper Family, Device, and Package for your project. This depends on the chip you are targeting for this project. The appropriate settings for a project suited for the Nexys2 500k board are as follows :



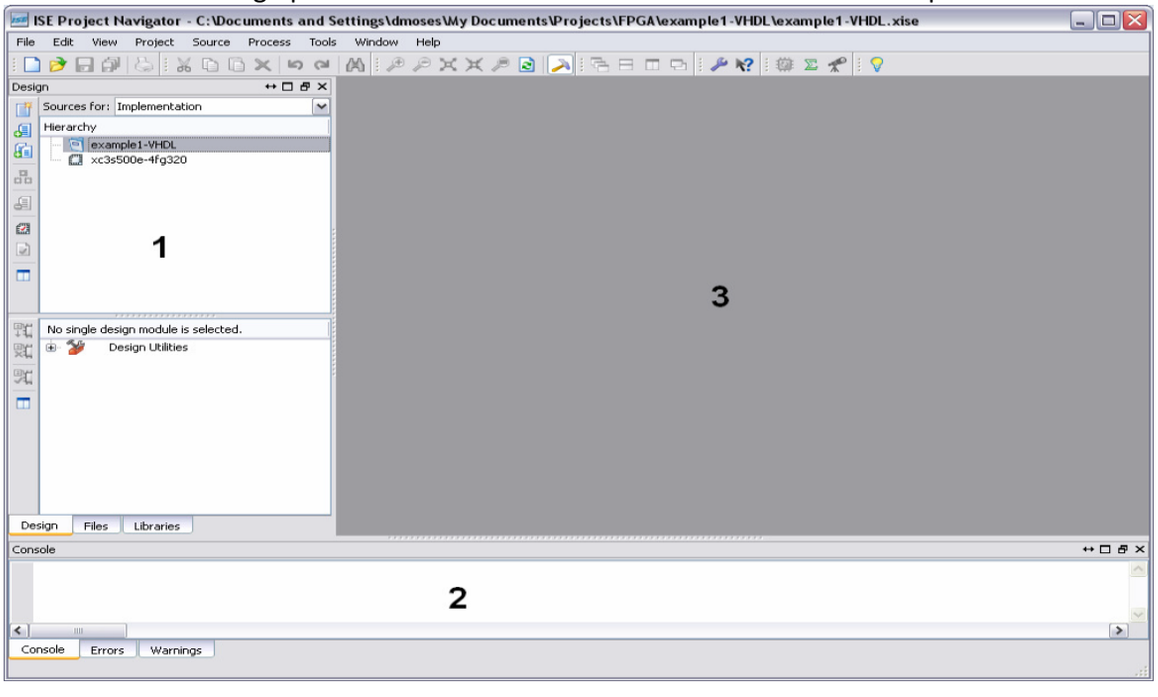
Since this lab does not require to physically burn the logic on a chip so we will leave this window as it is. Just click Next

3. Once the appropriate settings have been entered, click *Next*. The next two dialog boxes give you the option of adding new or existing source files to your project. Since we will fulfill these steps later, click *Next* without adding any source files.



9.3.2 Project Navigator Overview

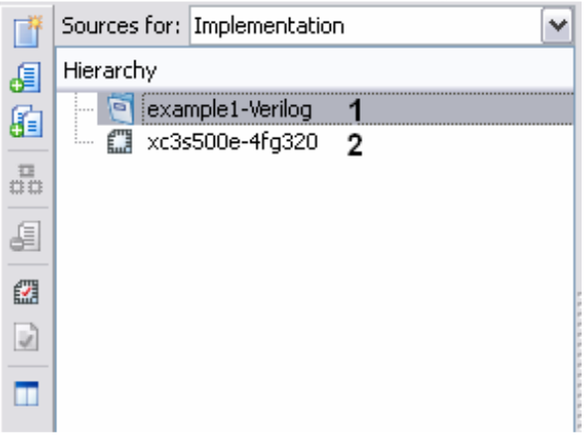
Once the new project has been created, ISE opens the project in Project Navigator. Click the Design tab to show the Design panel and click the Console tab to show the Console panel.



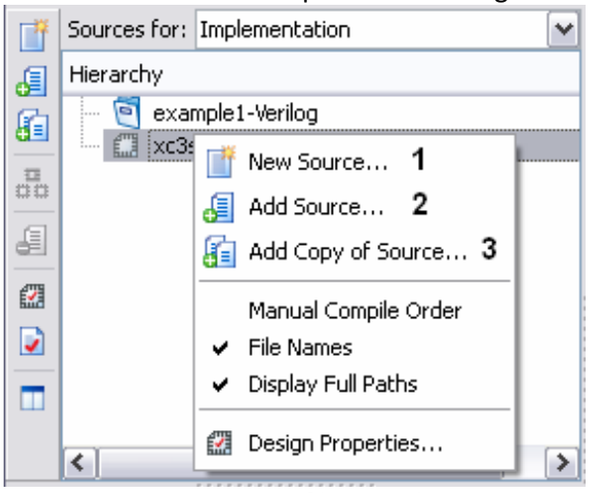
The Design panel (1) contains two windows: a Sources window that displays all source files associated with the current design and a Process window that displays all available processes that can be run on a selected source file.
The Console panel (2) displays status messages including error and warning messages.
The HDL editor window (3) displays source code from files selected in the Design panel.

9.3.3 Adding New Source Files

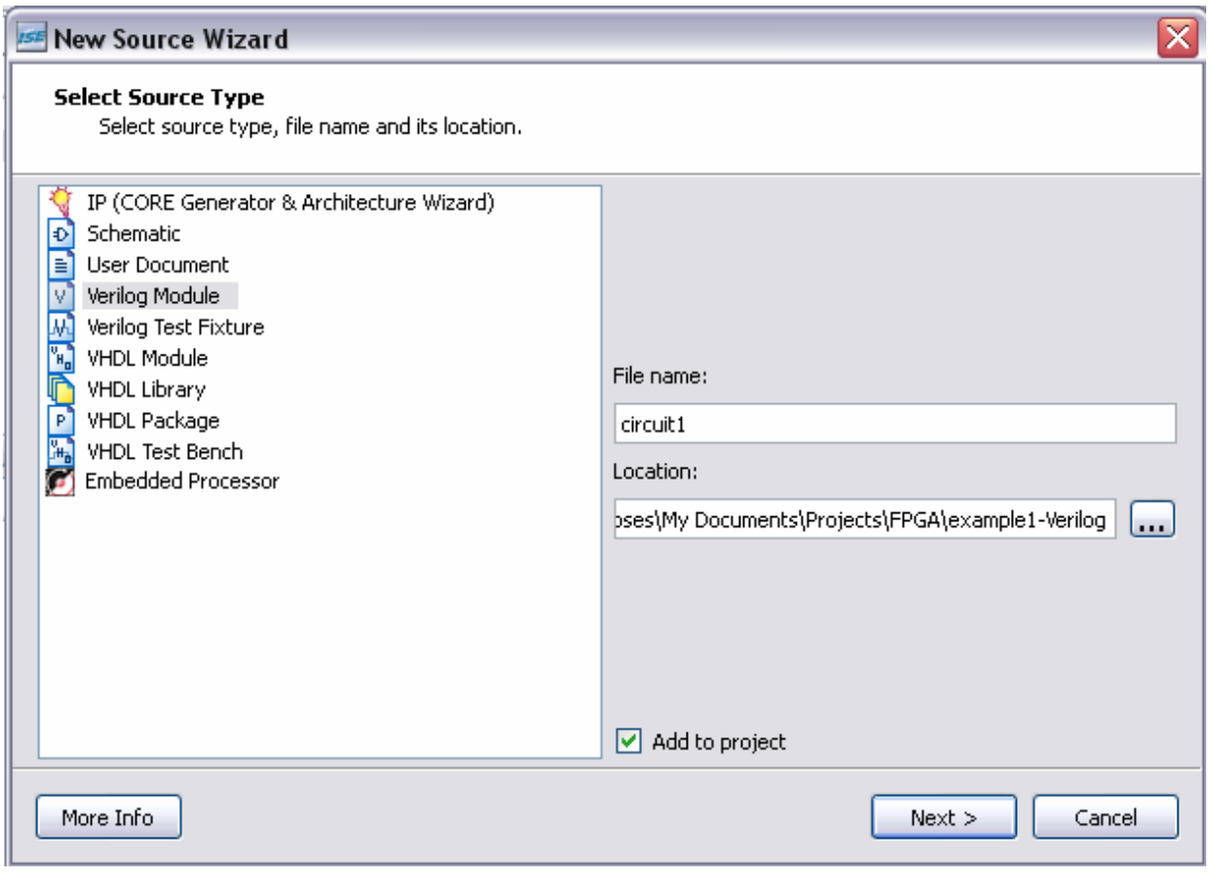
Once the new project is created, two sources are listed under sources in the Design panel: the Project file name and the Device targeted for design.



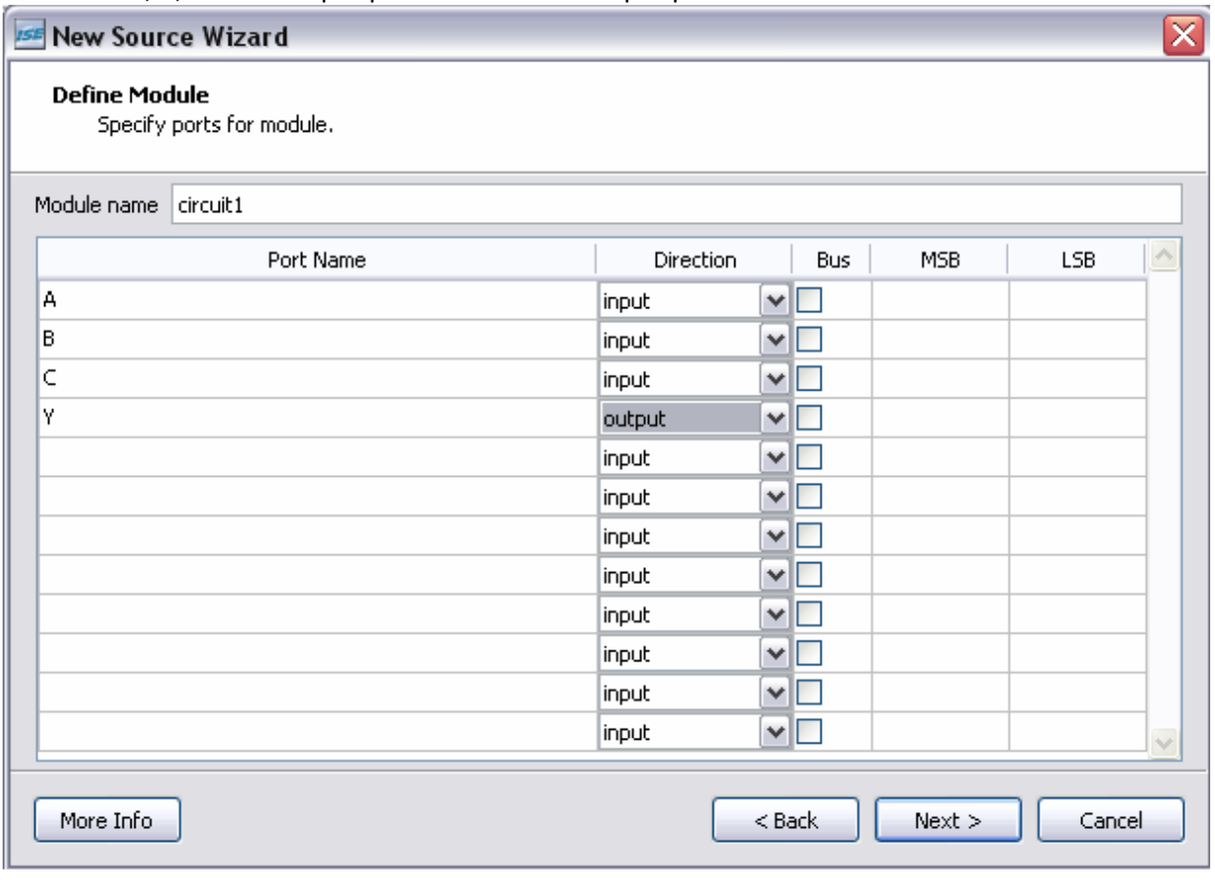
You can add a new or existing source file to the project. To do this, right-click the target device and select one of the three options for adding source files.



We create a new source file, so select New Source from the list. This starts the New Source Wizard, which prompts you for the Source type and file name. Select Verilog Module and give it a meaningful name e.g. circuit1

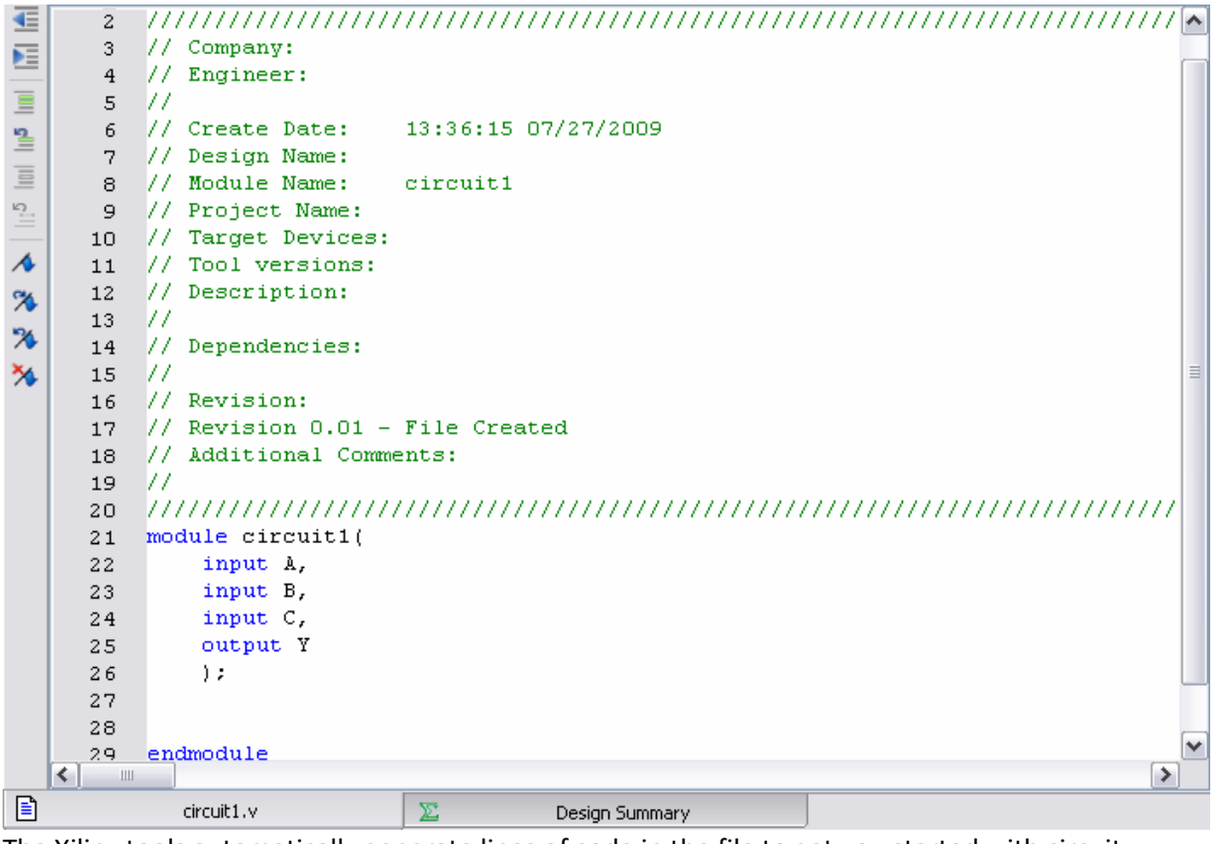


When you click Next, you have the option of defining top-level ports for the new Verilog module. We chose A, B, and C as input ports and Y as an output port.



9.3.4 HDL Editor Window

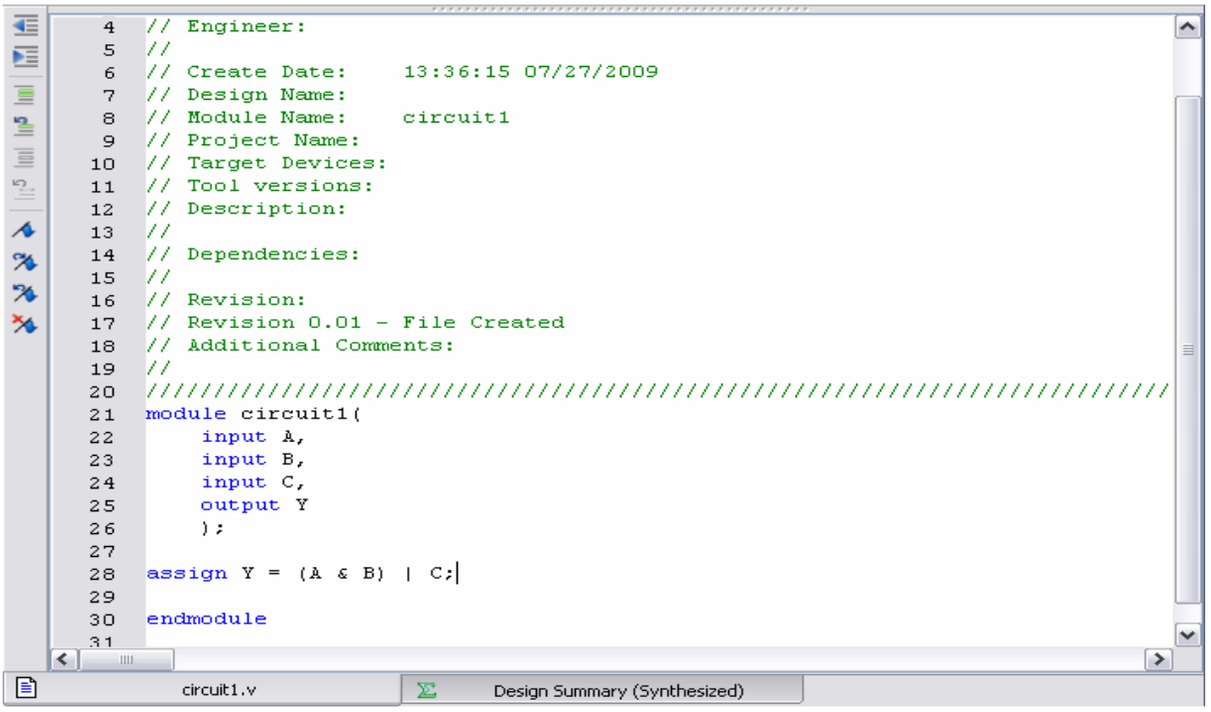
Once you have created the new Verilog file, the HDL Editor Window displays the circuit1.v source code.



The Xilinx tools automatically generate lines of code in the file to get you started with circuit development. This generated code includes:

1. a module statement
2. a comment block template for documentation

The actual behavioral or structural description of the given circuit is to be placed between the “module” and “endmodule” statements in the file. Between these statements, you can define any Verilog circuit you wish. In this tutorial, we use a simple combinational logic example, and then show how it can be used as a structural component in another Verilog module. We start with the basic logic equation: Y<= (A·B) + C.



If there are any syntax errors in the source file, ISE can help you find them. For example, the parentheses around the A and B inputs are crucial to this implementation; without them, ISE would return an error during synthesis info.

This Verilog module can now also be used as an instantiation statement in a separate project module. To illustrate this, we add another source file called “circuit2.v” and give it four input ports and one output port as follows:

```
module circuit2(
    input AT,
    input BT,
    input CT,
    input DT,
    output YT
    );
```

Instantiation statements often require the use of wires, which is a data type in Verilog that allows two points to be connected, like a real wire. A wire definition in Verilog must follow this format:

wire <wire name>;

The definition of the wire that we will use for this example is as follows:

wire sig;

The instantiation statement must follow the following format:

<name of module being instantiated> <arbitrary name>(.<instantiated component I/O>(<toplevel I/O or wire>));

The instantiation of the structural component circuit1 is as follows:

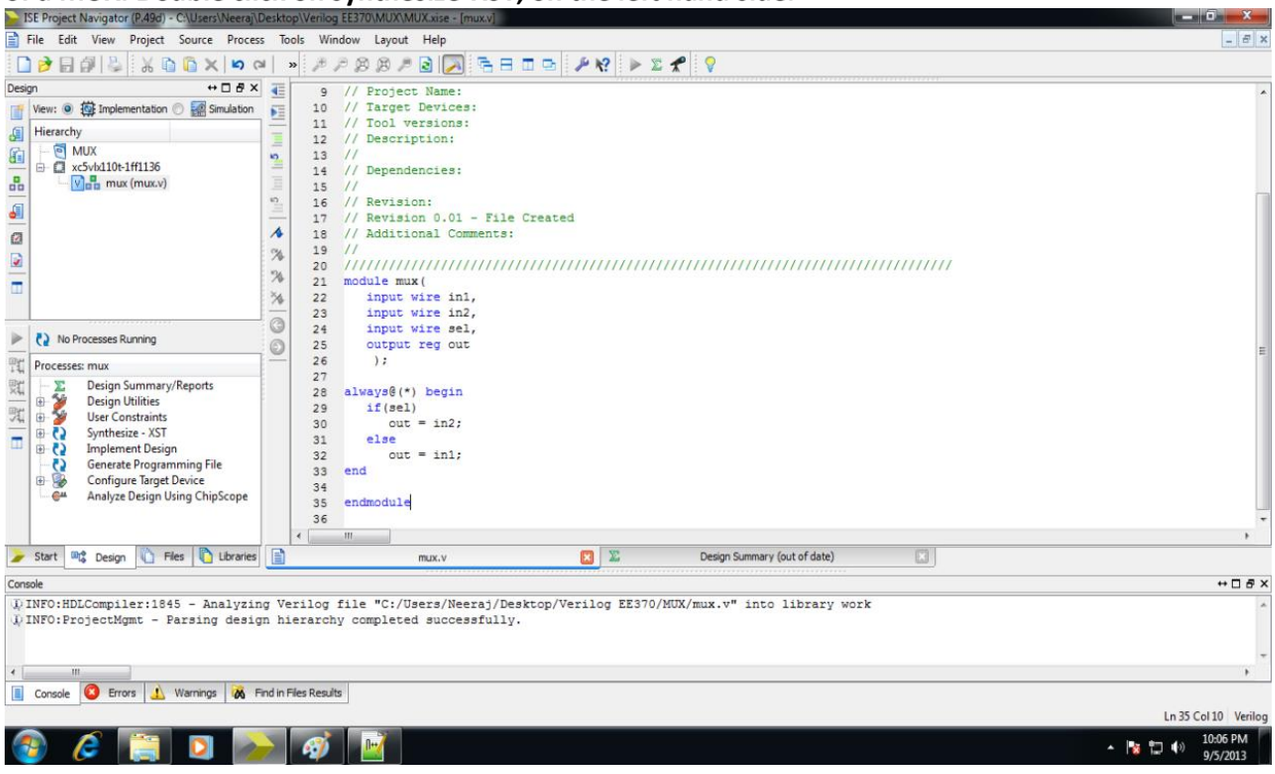
```
circuit1 C1 (.A(AT),
             .B(BT),
             .C(sig),
             .Y(YT));
```

To finish defining the simple circuit we will add the statement:
assign sig = CT & DT;
The finished definition of the circuit Y<= (A·B) + (C·D) is as follows:

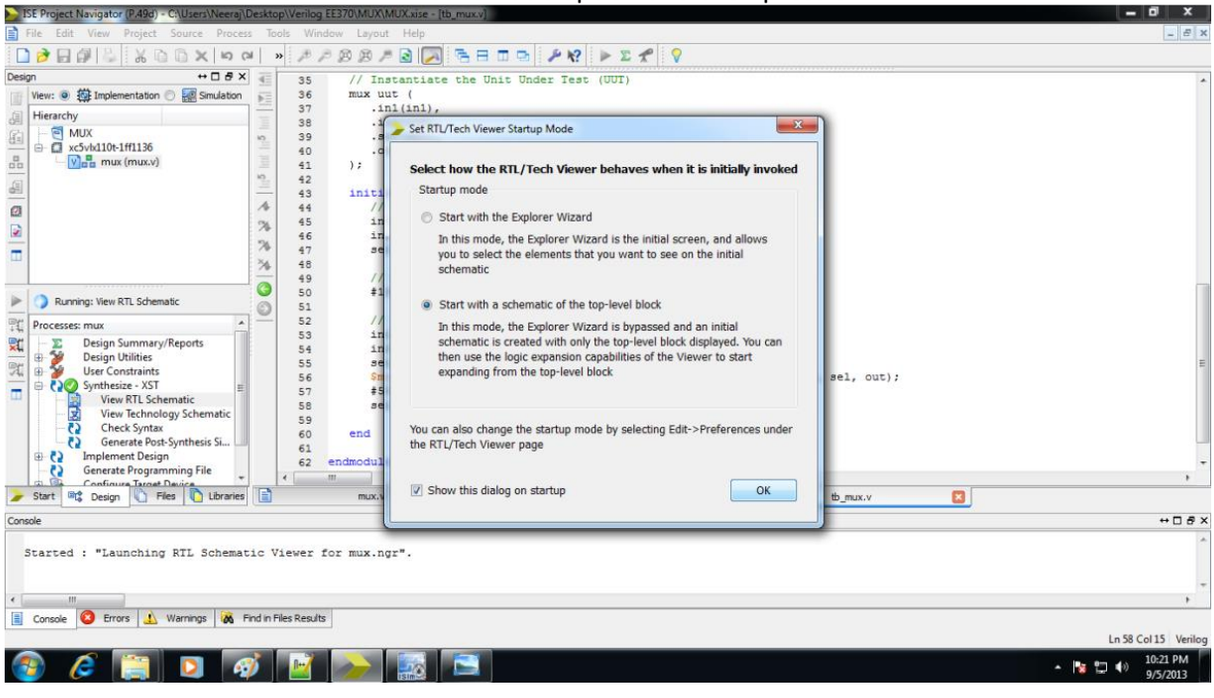
```
module circuit2(
    input AT,
    input BT,
    input CT,
    input DT,
    output YT
    );

    wire sig;

    circuit1 C1 (.A(AT),
                 .B(BT),
                 .C(sig),
                 .Y(YT));

   assign sig = CT & DT;

endmodule
```

9.3.5 RTL Schematics and Syntax Checking

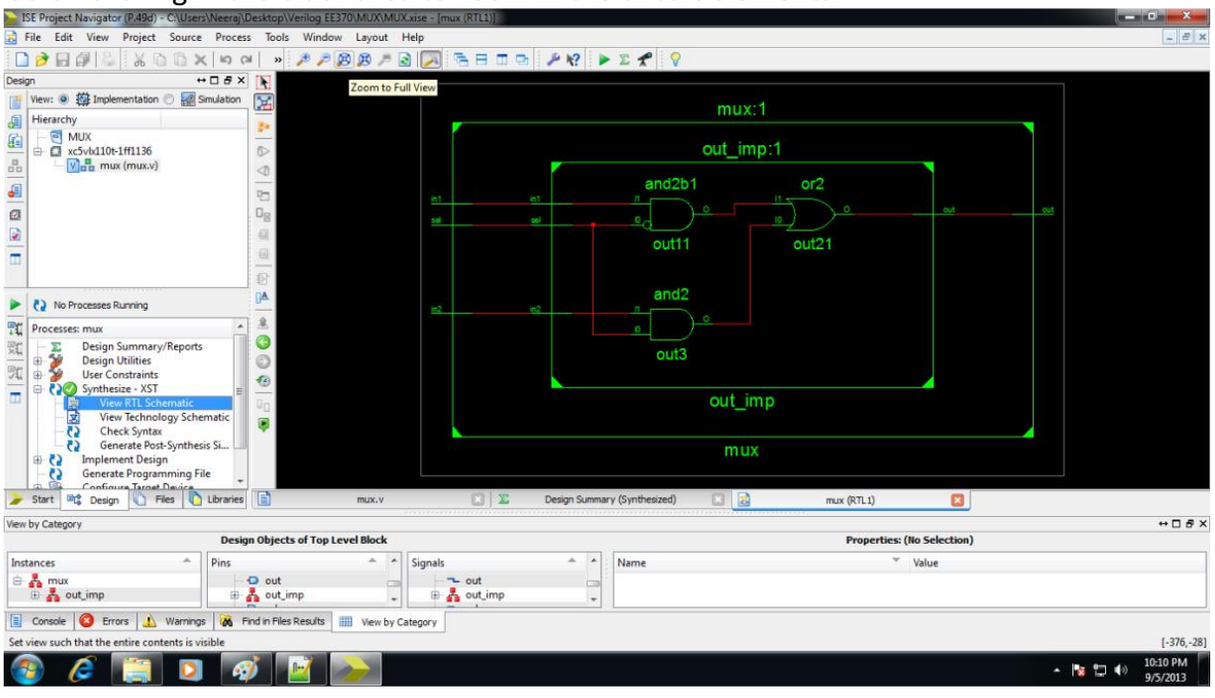
The RTL schematics can be viewed using the following set of steps, the example used here is that of a MUX. Double click on synthesize-XST, on the left hand side.



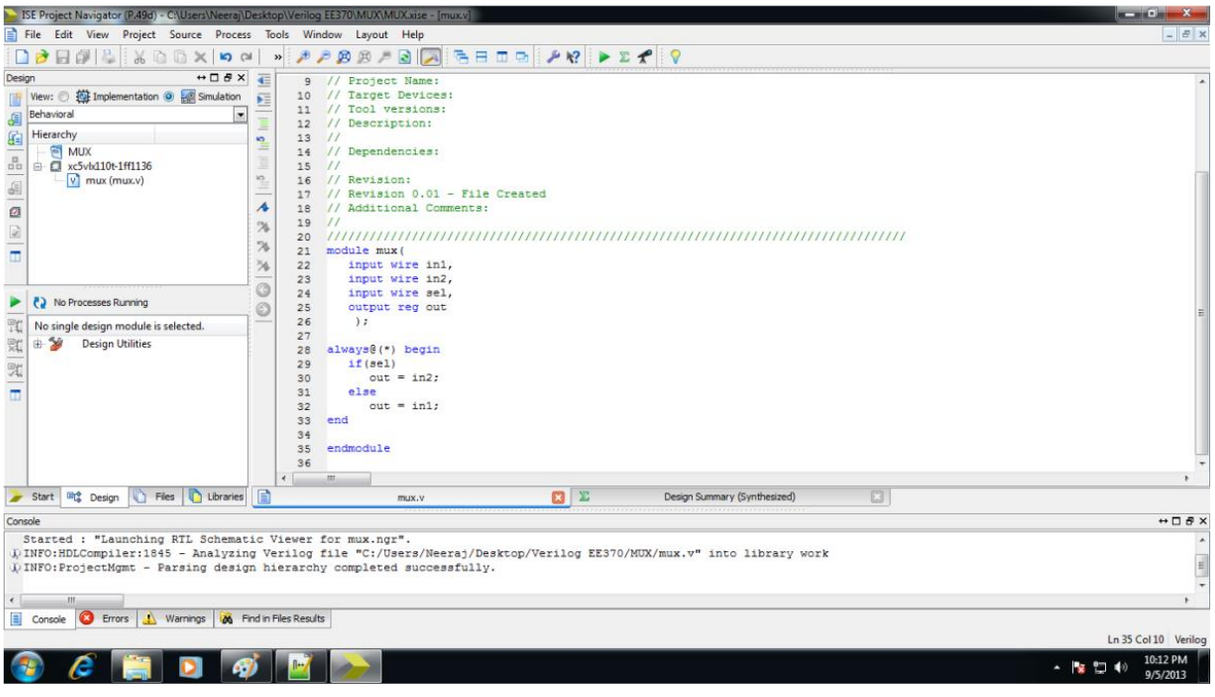
To see the circuit click on view RTL schematic option and then press ok.



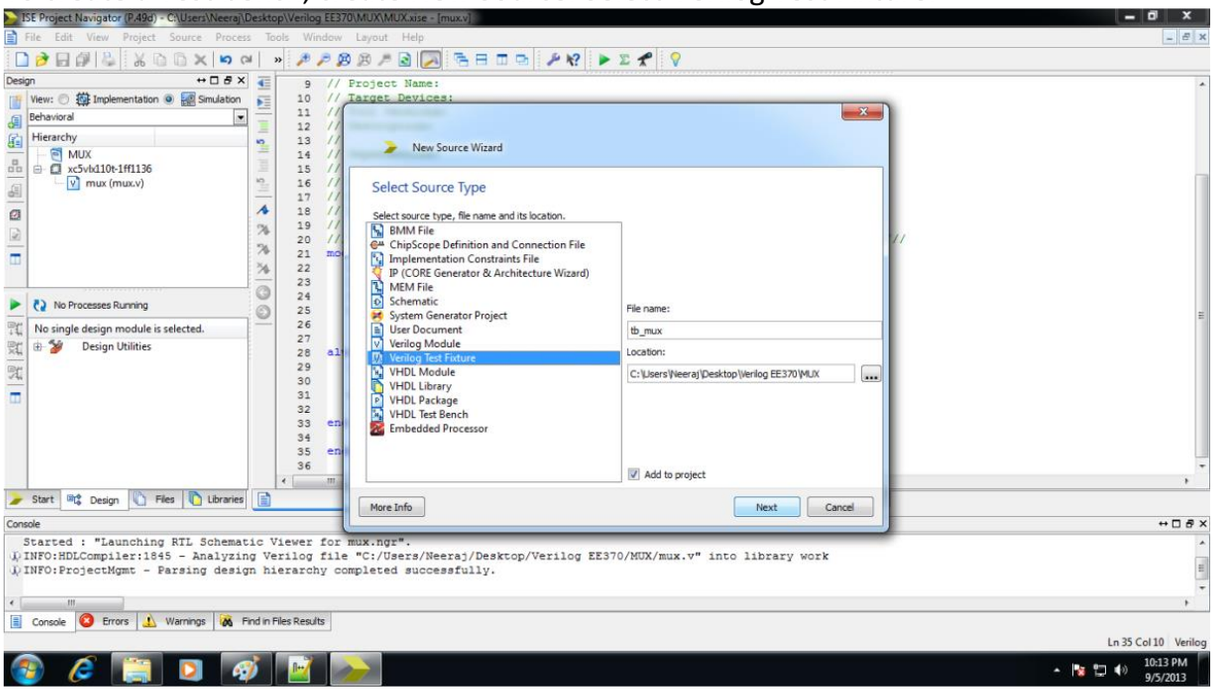
Go on clicking in the black area to zoom in the circuit elements.



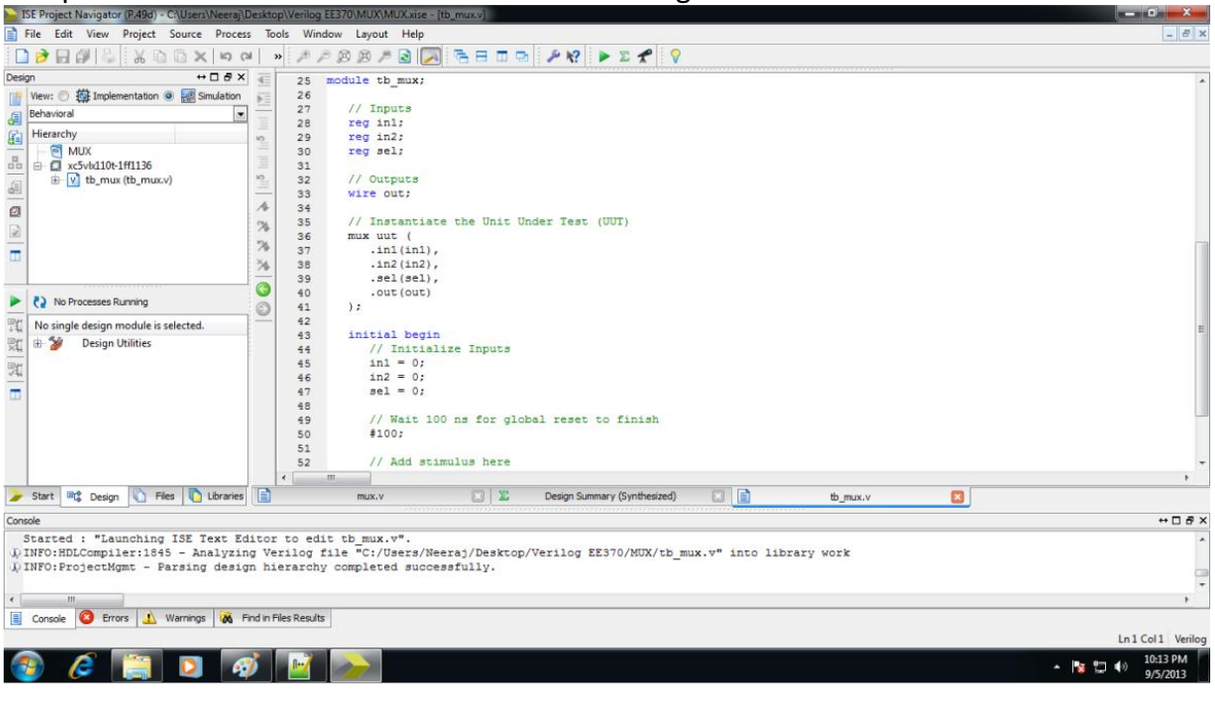
To run simulation click on “Simulation” option at the top of left column



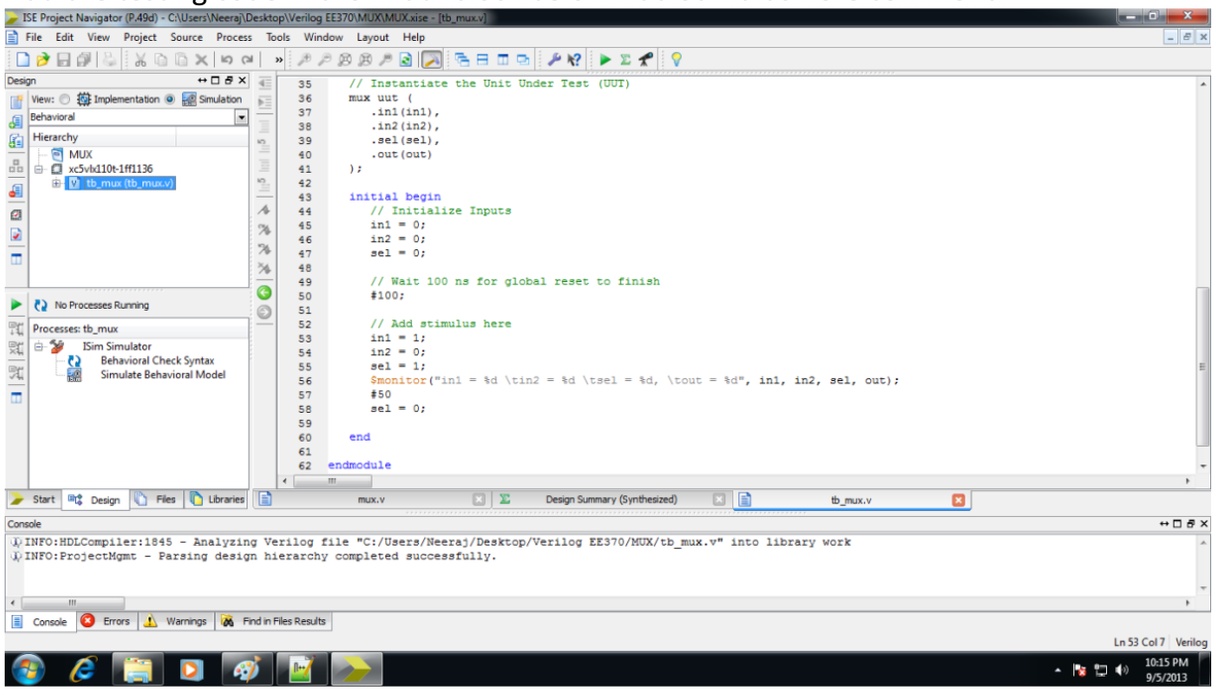
To create a Test bench, create New Source. Select Verilog Test Fixture.



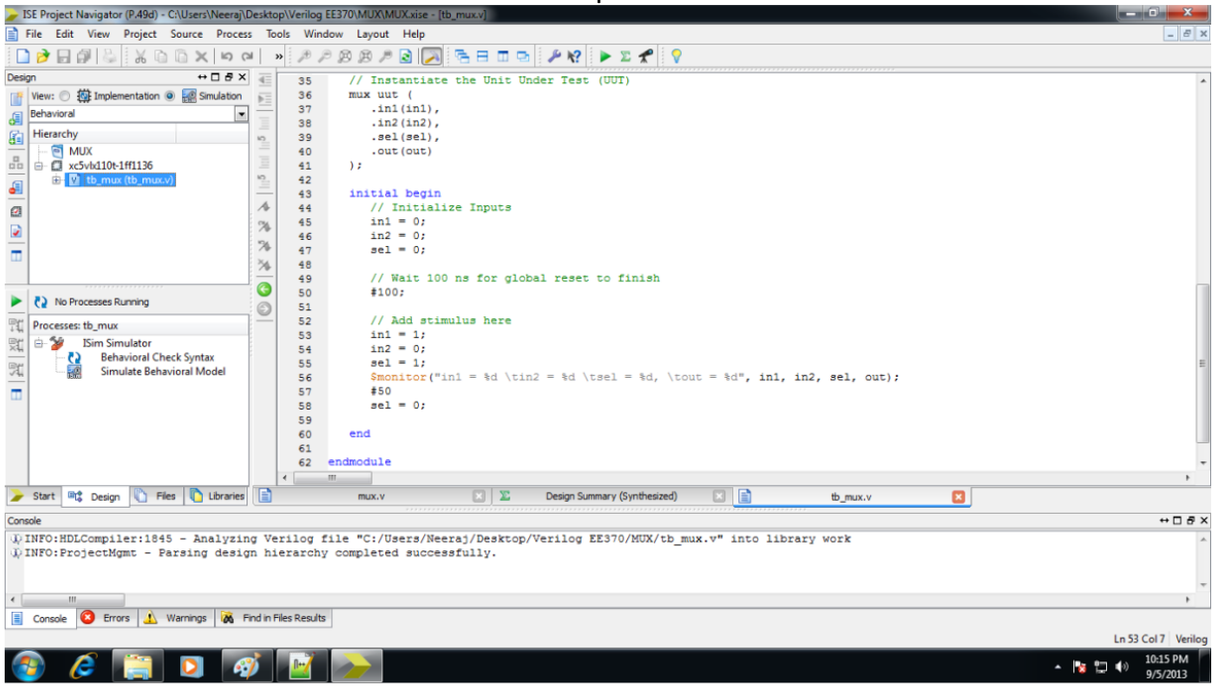
Template of Test bench will be created instantiating the mux module.



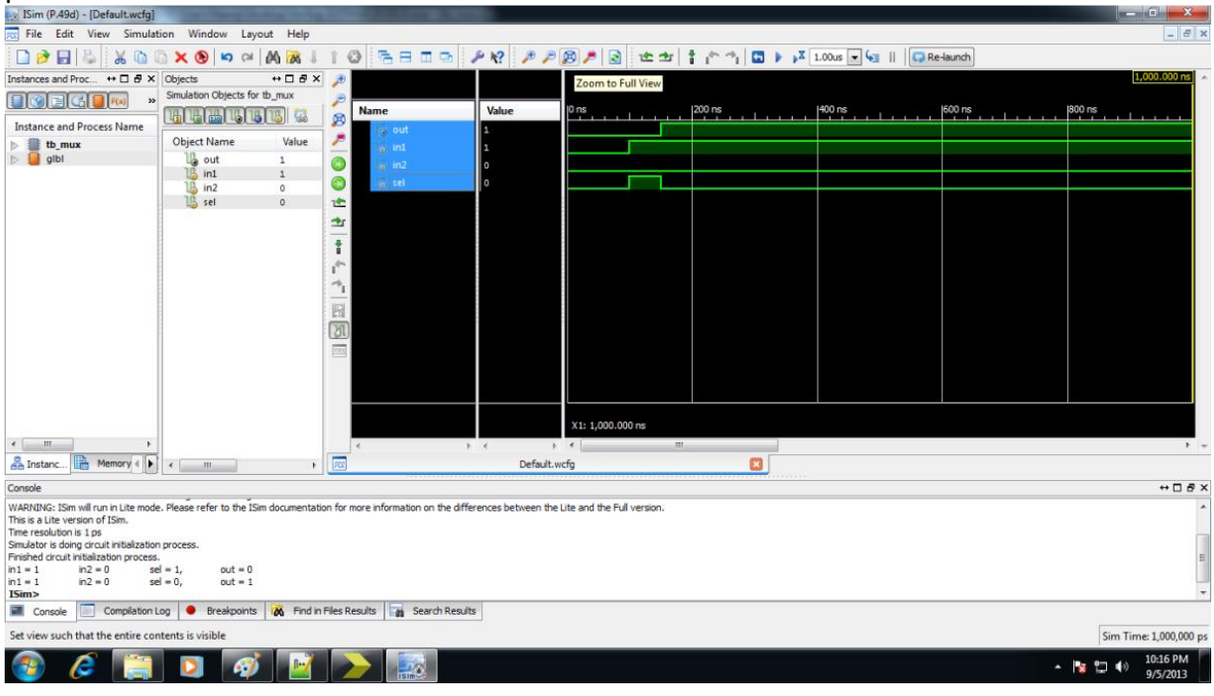
Add the testing code in the initial block below Add Stimulus here comment.



Double click on Simulate Behavioral Model option.



This is the simulation window. You can verify the working using waveforms or using printed statements at the bottom.



9.4 Verilog Codes (to be utilized in this lab) and task

Our assignment is to make a half-adder which involves two modules. The first module implements the half adder while the second one is the Stimulus to test it.

Main module (Half Adder)

```
module half_adder(sum, carry, in0, in1);
output sum, carry;
input in0, in1;
// 2-input XOR gate.
xor xor1(sum, in0, in1);
// 2-input AND gate.
and and1(carry, in0, in1);
endmodule
```

Stimulus

```
module Stimulus;
wire sum, carry;
reg in0, in1;
//Instantiation, will be discussed in more detail in future labs
half_adder hd1(sum, carry, in0, in1);
//Values checking part
initial
begin
in0 = 0; in1 = 0; // sum = 0, carry = 0
#10 in0 = 0; in1 = 1; // sum = 1, carry = 0
#10 in0 = 1; in1 = 0; // sum = 1, carry = 0
#10 in0 = 1; in1 = 1; // sum = 0, carry = 1
#10 $stop;
#10 $finish; end
endmodule
```

Lab Tasks will be provided at the end of this lab by the instructor.

Lab - 10: Modules, Ports, Data Types & Strings

10.1 HDL Coding

Verilog is both a behavioral and a structural language. Internals of each module can be defined at four levels of abstraction, depending on the needs of the design. The module behaves identically with the external environment irrespective of the level of abstraction at which the module is described. The internals of the module are hidden from the environment. Thus, the level of abstraction to describe a module can be changed without any change in the environment. The levels are defined below.

10.2 Behavioral or algorithmic level

This is the highest level of abstraction provided by Verilog HDL. A module can be implemented in terms of the desired design algorithm without concern for the hardware implementation details. Designing at this level is very similar to C programming.

10.3 Dataflow level

At this level, the module is designed by specifying the data flow. The designer is aware of how data flows between hardware registers and how the data is processed in the design.

10.3.1 Gate level

The module is implemented in terms of logic gates and interconnections between these gates. Design at this level is similar to describing a design in terms of a gate-level logic diagram.

10.3.2 Switch level

This is the lowest level of abstraction provided by Verilog. A module can be implemented in terms of switches, storage nodes, and the interconnections between them. Design at this level requires knowledge of switch-level implementation details.

Verilog allows the designer to mix and match all four levels of abstractions in a design. Due to increasing complexity of circuits, Switch Level Modeling is becoming rare so we will not discuss it in these labs.

10.4 Gate Level Modeling

In this lab, we discuss a design at a low level of abstraction—gate level. Most digital design is now done at gate level or higher levels of abstraction. At gate level, the circuit is described in terms of gates (e.g., and, nand). Hardware design at this level is intuitive for a user with a basic knowledge of digital logic design because it is possible to see a one-to-one correspondence between the logic circuit diagram and the Verilog description.

10.4.1 Gate Types

A logic circuit can be designed by use of logic gates. Verilog supports basic logic gates as predefined primitives. These primitives are instantiated like modules except that they are predefined in Verilog and do not need a module definition. All logic circuits can be designed by using basic gates. There are two classes of basic gates: and/or gates and buf/not gates.

a) And/ Or Gates

And/or gates have one scalar output and multiple scalar inputs. The first terminal in the list of gate terminals is an output and the other terminals are inputs. The output of a gate is evaluated as
soon as one of the inputs changes. The and/or gates available in Verilog are shown below.
and *or* *xor*
nand *nor* *xnor*
The corresponding logic symbols for these gates are shown in Figure 10.1. We consider gates with two inputs. The output terminal is denoted by out. Input terminals are denoted by i1 and i2.

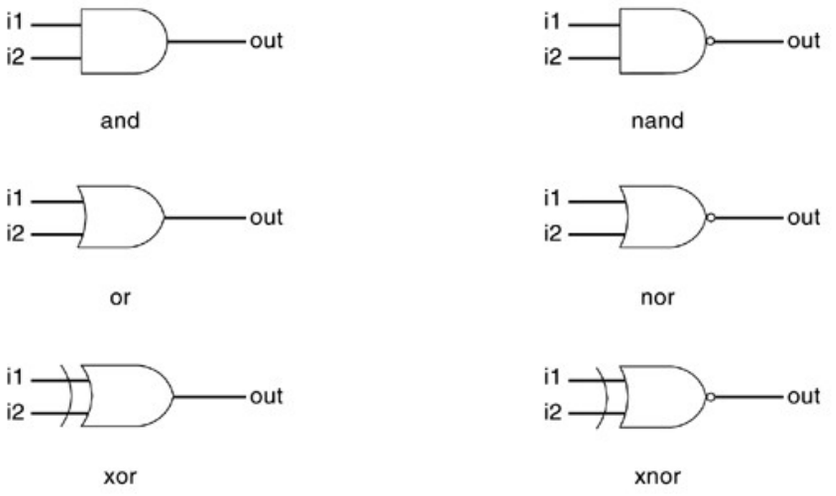


Figure 10.1 Basic Gates

These gates are instantiated to build logic circuits in Verilog. Examples of gate instantiations are shown below. Note that the instance name does not need to be specified for primitives. This lets the designer instantiate hundreds of gates without giving them a name. More than two inputs can be specified in a gate instantiation. Gates with more than two inputs are instantiated by simply adding more input ports in the gate instantiation. Verilog automatically instantiates the appropriate gate.

Gate Instantiation of And/Or Gates

```
wire OUT, IN1, IN2;

// basic gate instantiations

and a1(OUT, IN1, IN2);
nand na1(OUT, IN1, IN2);
or or1(OUT, IN1, IN2);
nor nor1(OUT, IN1, IN2);
xor x1(OUT, IN1, IN2);
xnor nx1(OUT, IN1, IN2); // More than two inputs;
```

3 input nand gate

```
nand na1_3inp(OUT, IN1, IN2, IN3);

// gate instantiation without instance name
and (OUT, IN1, IN2); // legal gate instantiation
```

The truth tables for these gates define how outputs for the gates are computed from the inputs. Truth tables are defined assuming two inputs. The truth tables for these gates are shown in the following Table. Outputs of gates with more than two inputs are computed by applying the truth table iteratively.

and	i1: 0	1	x	z
i2: 0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

nand	i1: 0	1	x	z
i2: 0	1	1	1	1
1	1	0	x	x
x	1	x	x	x
z	1	x	x	x

or	i1: 0	1	x	z
i2: 0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

nor	i1: 0	1	x	z
i2: 0	1	0	x	x
1	0	0	0	0
x	x	0	x	x
z	x	0	x	x

xor	i1: 0	1	x	z
i2: 0	0	1	x	x
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

xnor	i1: 0	1	x	z
i2: 0	1	0	x	x
1	0	1	x	x
x	x	x	x	x
z	x	x	x	x

Table 10.2: Truth Tables for And/Or Gates

Note: Values mentioned above can be read as:

Value Level	Condition in Hardware Circuits
0	Logic zero, false condition
1	Logic one, true condition
x	Unknown logic value
z	High impedance, floating state

Refer to this data-set for all truth tables.

b) Buf/Not Gates

Buf/not gates have one scalar input and one or more scalar outputs. The last terminal in the port list is connected to the input. Other terminals are connected to the outputs. We will discuss gates that have one input and one output.

Two basic buf/not gate primitives are provided in Verilog.

buf not

The symbols for these logic gates are shown in Figure 10.3.

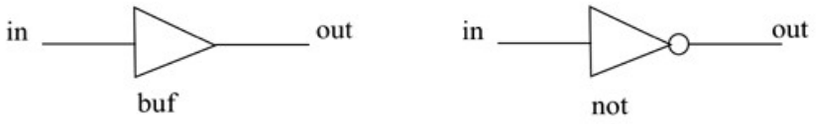


Figure 10.3: Buf and Not Gates

These gates are instantiated in Verilog as shown in the following examples. Notice that these gates can have multiple outputs but exactly one input, which is the last terminal in the port list.

Gate Instantiations of Buf/Not Gates

```
// basic gate instantiations
buf b1(OUT1, IN); not n1(OUT1, IN);
// More than two outputs
buf b1_2out(OUT1, OUT2, IN);
// gate instantiation without instance name
not (OUT1, IN); // legal gate instantiation
```

The truth tables for these gates are very simple. Truth tables for gates with one input and one output are shown in the following Table.

buf	in	out
	0	0
	1	1
	x	x
	z	x

not	in	out
	0	1
	1	0
	x	x
	z	x

Figure 10.4: Truth Tables for Buf/Not Gates

c) Bufif/notif

Gates with an additional control signal on buf and not gates are also available.
bufif1 notif1
bufif0 notif0
These gates propagate only if their control signal is asserted. They propagate z if their control signal is de-asserted. Symbols and truth tables for bufif/notif are shown in Figure 10.5 and Figure 10.6 respectively.

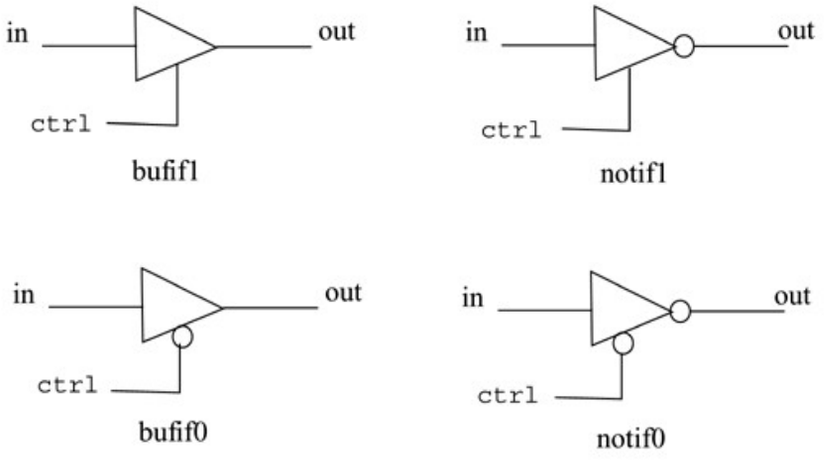


Figure 10.5: Gates Bufif and Notif

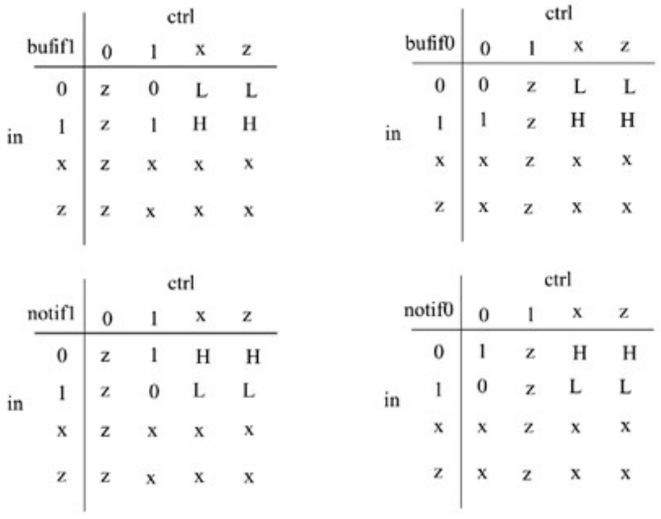


Figure 10.6: Truth Tables for Bufif/Notif Gates

These gates are used when a signal is to be driven only when the control signal is asserted. Such a situation is applicable when multiple drivers drive the signal. These drivers are designed to drive the signal on mutually exclusive control signals. Following examples show instantiation of bufif and notif gates.

Gate Instantiations of Bufif/Notif Gates

//Instantiation of bufif gates.

bufif1 b1 (out, in, ctrl);

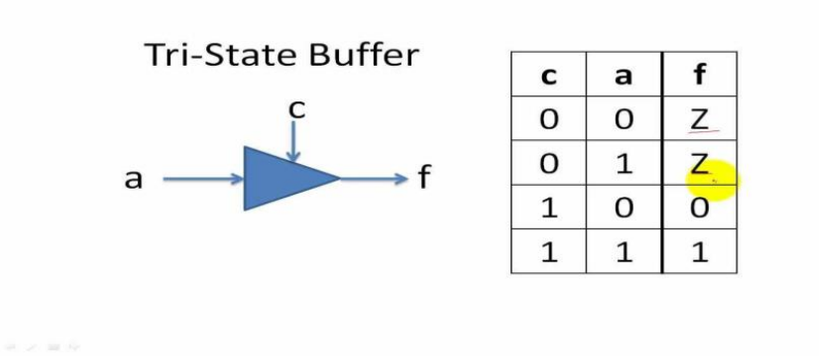
bufif0 b0 (out, in, ctrl);

//Instantiation of notif gates

notif1 n1 (out, in, ctrl);

notif0 n0 (out, in, ctrl);

Tri State Buffer



c	a	f
0	0	Z
0	1	Z
1	0	0
1	1	1

Figure 10.7: Truth Table for Tri-State Buffer

Main module

```
module tristate_buffer (data_out, data_in, enable);
   output data_out;
   input  data_in;
   input  enable;

   assign data_out = enable ? data_in : 1'bz;

endmodule
```

Verilog Codes and task:

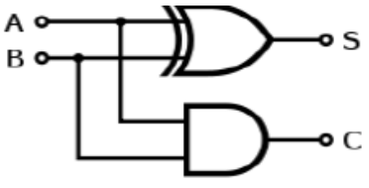


Figure 10.8: Half-Adder

The logic diagram of a half-adder is given above. It consists of two gates. One XOR and one AND gate. The truth table is as follows:

Input		Output	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure 10.9: Truth Tables for Half-Adder

10.6.1 Main module (Half Adder)

```
module half_adder(S, C, A, B);
output S, C;
input A, B;
// 2-input XOR gate.
xor xor1(S, A, B);
// 2-input AND gate.
and and1(C, A, B);
endmodule
```

10.6.2 Stimulus

```
module Stimulus;
wire S, C;
reg A, B;
//Instantiation is the process of defining larger modules using smaller modules
half_adder hd1(S, C, A, B);
//Values checking part
initial
begin
A = 0; B = 0; // S = 0, C =0
#10 A = 0; B = 1; // S = 1, C =0
#10 A = 1; B = 0; // S = 1, C =0
#10 A = 1; B = 1; // S = 0, C =1
#10 $stop;
#10 $finish;
end
endmodule
```

Lab Task:

Lab Tasks will be provided at the end of this lab by the instructor.

Lab - 11: Dataflow Modeling

11.1 Introduction

For small circuits, the gate-level modeling approach works very well because the number of gates is limited and the designer can instantiate and connect every gate individually. However, in complex designs the number of gates is very large. Thus, designers can design more effectively if they concentrate on implementing the function at a level of abstraction higher than gate level. Dataflow modeling provides a powerful way to implement a design. Verilog allows a circuit to be designed in terms of the data flow between registers and how a design processes data rather than instantiation of individual gates.

11.1.1 Continuous Assignments

A continuous assignment is the most basic statement in dataflow modeling, used to drive a value onto a net. This assignment replaces gates in the description of the circuit and describes the circuit at a higher level of abstraction. The assignment statement starts with the keyword assign. The syntax of an assign statement is as follows.

continuous_assign::= assign [drive_strength] [delay3]
list_of_net_assignments;
list_of_net_assignments::= net_assignment { , net_assignment }
net_assignment ::= net_lvalue = expression

Examples of Continuous Assignment

```
// Continuous assign. out is a net. i1 and i2 are nets.
assign out = i1 & i2;
// Continuous assign for vector nets. addr is a 16-bit vector net
// addr1 and addr2 are 16-bit vector registers.
assign addr[15:0] = addr1_bits[15:0] ^ addr2_bits[15:0];
// Concatenation. Left-hand side is a concatenation of a scalar
// net and a vector net.
assign {c_out, sum[3:0]} = a[3:0] + b[3:0] + c_in;
```

a) Implicit Continuous Assignment

Instead of declaring a net and then writing a continuous assignment on the net, Verilog provides a shortcut by which a continuous assignment can be placed on a net when it is

declared. There can be only one implicit declaration assignment per net because a net is declared only once.

In the example below, an implicit continuous assignment is contrasted with a regular continuous assignment.

b) Regular Continuous Assignment

```
wire out;
assign out = in1 & in2;
//Same effect is achieved by an implicit continuous assignment
wire out = in1 & in2;
```

c) Implicit Net Declaration

If a signal name is used to the left of the continuous assignment, an implicit net declaration will be inferred for that signal name. If the net is connected to a module port, the width of the inferred net is equal to the width of the module port.

```
// Continuous assign. out is a net.
wire i1, i2;
assign out = i1 & i2;    //Note that out was not declared as a wire
                         //but an implicit wire declaration for out
                         //is done by the simulator
```

11.1.2 Expressions, Operators, and Operands

Dataflow modeling describes the design in terms of expressions instead of primitive gates. Expressions, operators, and operands form the basis of dataflow modeling.

a) Expressions

Expressions are constructs that combine operators and operands to produce a result.

```
// Examples of expressions. Combines operands and operators
a ^ b
addr1[20:17] + addr2[20:17] in1 | in2
```

b) Operands

Operands can be constants, integers, real numbers, nets, registers, times, bit-select (one bit of vector net or a vector register), part-select (selected bits of the vector net or register vector), and memories or function calls (functions are discussed later).

```
integer count, final_count;
final_count = count + 1; //count is an integer operand
real a, b, c;
c = a - b;          //a and b are real operands

reg [15:0] reg1, reg2; reg [3:0] reg_out;
reg_out = reg1[3:0] ^ reg2[3:0]; //reg1[3:0] and reg2[3:0] are
                                 //part-select register operands
reg ret_value;
```

ret_value = calculate_parity(A, B); **//calculate_parity is a**
//function type operand

c) Operators

Operators act on the operands to produce desired results. Verilog provides various types of operators.

d1 && d2 // **&& is an operator on operands d1 and d2**
!a[0] // **! is an operator on operand a[0]**
B >> 1 // **>> is an operator on operands B and 1**

11.1.3 Operator Types

Verilog provides many different operator types. Operators can be arithmetic, logical, relational, equality, bitwise, reduction, shift, concatenation, or conditional.

Operator Type	Operator Symbol	Operation Performed	Number of Operands
Arithmetic	* / + - % **	Multiply divide add subtract modulus power (exponent)	two two two two two two
Logical	! && ||	logical negation logical and logical or	one two two
Relational	> < >= <=	greater than less than greater than or equal less than or equal	two two two two
Equality	== != === !==	Equality inequality case equality case inequality	two two two two
Bitwise	~ & | ^ ^~ or ~^	bitwise negation bitwise and bitwise or bitwise xor bitwise xnor	one two two two two
Reduction	& ~& | ~| ^ ^~ or ~^	reduction and reduction nand reduction or reduction nor reduction xor reduction xnor	one one one one one one
Shift	>> << >>> <<<	Right shift Left shift Arithmetic right shift Arithmetic left shift	two two two two
Concatenation	{ }	Concatenation	any number
Replication	{ { } }	Replication	Any number
Conditional	?:	Conditional	Three

Table 11.1: Operator Types and Symbols

a) Shift Operators

Shift operators are right shift (>>), left shift (<<), arithmetic right shift (>>>), and arithmetic left shift (<<<). Regular shift operators shift a vector operand to the right or the left by a specified number of bits. The operands are the vector and the number of bits to shift. When the bits are shifted, the vacant bit positions are filled with zeros. Shift operations do not wrap around. Arithmetic shift operators use the context of the expression to determine the value with which to fill the vacated bits.

```
// X = 4'b1100

Y = X >> 1; //Y is 4'b0110. Shift right 1 bit. 0 filled in MSB position.
Y = X << 1; //Y is 4'b1000. Shift left 1 bit. 0 filled in LSB position.
Y = X << 2; //Y is 4'b0000. Shift left 2 bits.
integer a, b, c; //Signed data types
a = 0;
b = -10; // 00111...10110 binary
c = a + (b >>> 3); //Results in -2 decimal, due to arithmetic shift
```

Shift operators are useful because they allow the designer to model shift operations, shift-and- add algorithms for multiplication, and other useful operations.

b) Concatenation Operator

The concatenation operator ({, }) provides a mechanism to append multiple operands. The operands must be sized. Unsized operands are not allowed because the size of each operand must be known for computation of the size of the result.

Concatenations are expressed as operands within braces, with commas separating the operands. Operands can be scalar nets or registers, vector nets or registers, bit-select, part-select, or sized constants.

```
// A = 1'b1, B = 2'b00, C = 2'b10, D = 3'b110
Y = {B , C} // Result Y is 4'b0010
Y = {A , B , C , D , 3'b001} // Result Y is 11'b10010110001
Y = {A , B[0], C[1]} // Result Y is 3'b101
```

c) Replication Operator

Repetitive concatenation of the same number can be expressed by using a replication constant. A replication constant specifies how many times to replicate the number inside the brackets ({ }).

```
reg A;
reg [1:0] B, C;
reg [2:0] D;
A = 1'b1; B = 2'b00; C = 2'b10; D = 3'b110;
Y = { 4{A} } // Result Y is 4'b1111
Y = { 4{A} , 2{B} } // Result Y is 8'b11110000
Y = { 4{A} , 2{B} , C } // Result Y is 8'b1111000010
```

d) Conditional Operator

The conditional operator(?:) takes three operands. The syntax of an assign statement is as follows.
condition_expr ? true_expr : false_expr ;

The condition expression (condition_expr) is first evaluated. If the result is true (logical 1), then the true_expr is evaluated. If the result is false (logical 0), then the false_expr is evaluated. If the result is x (ambiguous), then both true_expr and false_expr are evaluated and their results are compared, bit by bit, to return for each bit position an x if the bits are different and the value of the bits if they are the same.
The action of a conditional operator is similar to a multiplexer. Alternately, it can be compared to the if-else expression. Conditional operators are frequently used in dataflow modeling to model conditional assignments. The conditional expression acts as a switching control.

//model functionality of a 2-to-1 mux
assign out = control ? in1 : in0;

11.1.4 Operator Precedence

If no parentheses are used to separate parts of expressions, Verilog enforces the following precedence. It is recommended that parentheses be used to separate expressions except in case of unary operators or when there is no ambiguity.

Operators	Operator Symbols	Precedence
Unary	+ - ! ~	Highest precedence
Multiply, Divide, Modulus	* / %	
Add, Subtract	+ -	
Shift	<< >>	
Relational	< <= > >=	
Equality	== != === !==	
Reduction	&, ~& ^ ^~ |, ~|	
Logical	&& ||	
Conditional	?:	Lowest precedence

Table 11.2: Operator Precedence

11.2 Verilog Codes and task:

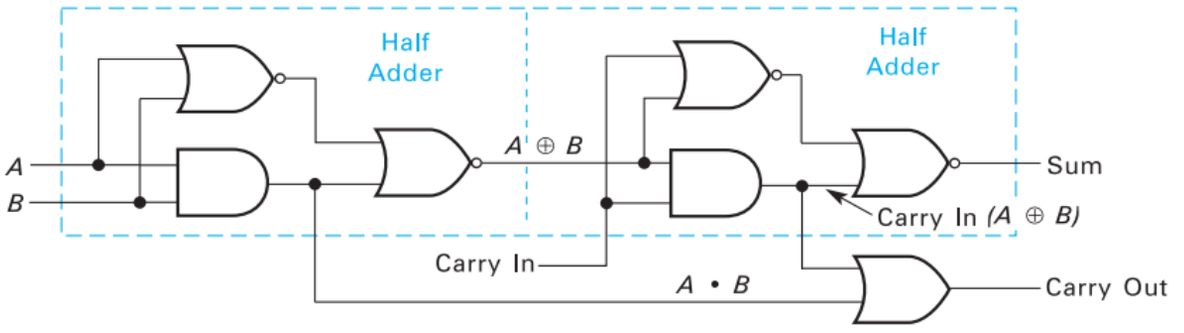


Figure 11.3: Full-Adder

The logic diagram of a full adder is given above. From above figure, it is clear that we can make a full adder using two **half adders** & an **OR gate**.

Half Adder

```
module half_adder(S, C, A, B);
output S, C;
input A, B;

//Gates have been replaced with an assign statement
assign {C,S} = A+B;

endmodule
```

Top Module (Full Adder)

```
module half_adder(Sum, CarryOut, A, B, CarryIn);
output Sum, CarryOut;
input A, B, CarryIn;
wire TSum, TCarry1, TCarry2;

//Two half-adders are instantiated. (This is similar to a function call in C)
half_adder ha1(TSum, TCarry1, A, B);
half_adder ha2(Sum, TCarry2, TSum, CarryIn);

//Or gate can be written as this assign statement
assign CarryOut = TCarry1 || TCarry2;
endmodule
```

Stimulus

```
module Stimulus;
wire Sum, CarryOut;
reg A, B, CarryIn;
```

```
//This time we have instantiated full-adder module as it is the one we are testing
full_adder fa1(Sum, CarryOut, A, B, CarryIn);
        //Values checking part

        initial begin
        A = 0; B = 0; CarryIn = 0; //Sum = 0, CarryOut = 0
        #10 A = 0; B = 0; CarryIn = 1; //Sum = 1, CarryOut = 0
        #10 A = 0; B = 1; CarryIn = 0; //Sum = 1, CarryOut = 0
        #10 A = 0; B = 1; CarryIn = 1; //Sum = 0, CarryOut = 1
        #10 A = 1; B = 0; CarryIn = 0; //Sum = 1, CarryOut = 0
        #10 A = 1; B = 0; CarryIn = 1; //Sum = 0, CarryOut = 1
        #10 A = 1; B = 1; CarryIn = 0; //Sum = 0, CarryOut = 1
        #10 A = 1; B = 1; CarryIn = 1; //Sum = 1, CarryOut = 1
        #10 $stop;
        #10 $finish;
        end

        endmodule
```

4-to1 multiplexer

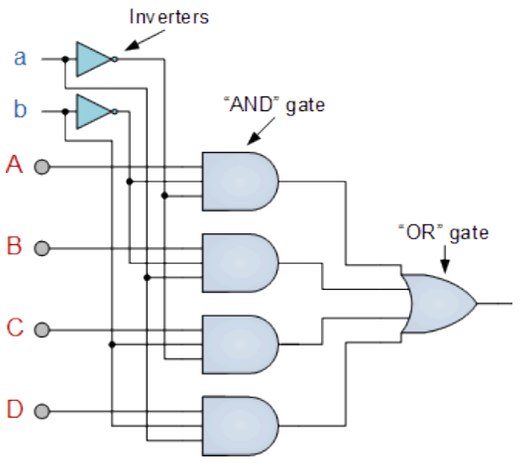


Figure 11.4: 4-to-1 Mux

Select Data Inputs		Output
S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Figure 11.5: 4-to-1 Mux Truth Table

```
module multiplexer4_1 (cout, a ,b ,c ,d ,x ,y );

        output cout ;
        input a ;
        input b ;
        input c ;
        input d ;
        input x ;
        input y ;

assign cout = (a & (~x) & (~y)) |
  (b & (~x) & (y)) |
  (c & x & (~y)) |
  (d & x & y);

endmodule
```

Alternative Code

```
module mux_4to1_assign ( input [3:0] a, // 4-bit input called a
        input [3:0] b, // 4-bit input called b
        input [3:0] c, // 4-bit input called c
        input [3:0] d, // 4-bit input called d
        input [1:0] sel, // input sel used to select between a,b,c,d
        output [3:0] out); // 4-bit output based on input sel
```

- `// When sel[1] is 0, (sel[0]? b:a) is selected and when sel[1] is 1, (sel[0] ? d:c) is taken`
- `// When sel[0] is 0, a is sent to output, else b and when sel[0] is 0, c is sent to output, else d`

```
        assign out = sel[1] ? (sel[0] ? d : c) : (sel[0] ? b : a);
endmodule
```

Lab Task

Lab Tasks will be provided at the end of this lab by the instructor.

Lab - 12: Verilog Behavioral Modeling

12.1 Introduction

Verilog provides designers the ability to describe design functionality in an algorithmic manner. In other words, the designer describes the **behavior** of the circuit. Thus, behavioral modeling represents the circuit at a very high level of abstraction. Design at this level resembles C programming more than it resembles digital circuit design. Behavioral Verilog constructs are similar to C language constructs in many ways. Verilog is rich in behavioral constructs that provide the designer with a great amount of flexibility.

12.1.1 Structured Procedures

There are two structured procedure statements in Verilog: *always* and *initial*. These statements are the two most basic statements in behavioral modeling. All other behavioral statements can appear only inside these structured procedure statements.

Verilog is a concurrent programming language unlike the C programming language, which is sequential in nature. Activity flows in Verilog run in parallel rather than in sequence. Each always and initial statement represents a separate activity flow in Verilog. Each activity flow starts at simulation time 0. The statements always and initial cannot be nested. The fundamental difference between the two statements is explained in the following sections.

a) Initial Statement

All statements inside an initial statement constitute an **initial block**. An initial block starts at time 0, executes exactly once during a simulation, and then does not execute again. If there are multiple initial blocks, each block starts to execute concurrently at time 0. Each block finishes execution independently of other blocks. Multiple behavioral statements must be grouped; typically using the keywords begin and end. If there is only one behavioral statement, grouping is not necessary. This is similar to the { } grouping in the C programming language.

Example:

```
initial
m = 1'b0; //single statement; does not need to be grouped
initial begin
#5 a = 1'b1; //multiple statements; need to be grouped
#25 b = 1'b0;
end
initial begin
#10 x = 1'b0;
#25 y = 1'b1;
```

```
end
initial
#50 $finish;
```

In the above example, the three initial statements start to execute in parallel at time 0. If a delay #<delay> is seen before a statement, the statement is executed <delay> time units after the current simulation time. Thus, the execution sequence of the statements inside the initial blocks will be as follows.

time	statement executed 0 m = 1'b0;
5	a = 1'b1;
10	x = 1'b0;
30	b = 1'b0;
35	y = 1'b1;
50	$finish;

The initial blocks are typically used for initialization, monitoring, waveforms and other processes that must be executed only once during the entire simulation run.

b) Always Statement

All behavioral statements inside an always statement constitute an **always block**. The always statement starts at time 0 and executes the statements in the always block continuously in a looping fashion. This statement is used to model a block of activity that is repeated continuously in a digital circuit. An example is a clock generator module that toggles the clock signal every half cycle. In real circuits, the clock generator is active from time 0 to as long as the circuit is powered on.

Example:

```
//Initialize clock at time zero
initial
clock = 1'b0;
//Toggle clock every half-cycle (time period = 20)
always
#10 clock = ~clock;
initial
#1000 $finish;
```

In the example, the always statement starts at time 0 and executes the statement clock = ~clock every 10 time units. Notice that the initialization of clock has to be done inside a separate initial statement. If we put the initialization of clock inside the always block, clock will be initialized every time the always is entered. Also, the simulation must be halted inside an initial statement. If there is no $stop or $finish statement to halt the simulation, the clock generator will run forever.

C programmers might draw an analogy between the always block and an infinite loop. But hardware designers tend to view it as a continuously repeated activity in a digital circuit starting from power on. The activity is stopped only by power off ($finish) or by an interrupt ($stop).

12.1.2 Procedural Assignment

Procedural assignments update values of reg, integer, real, or time variables. The value placed on a variable will remain unchanged until another procedural assignment updates the variable with a different value. These are unlike continuous assignments discussed in Dataflow Modeling, where one assignment statement can cause the value of the right-hand-side expression to be continuously placed onto the left-hand-side net. The syntax for the simplest form of procedural assignment is shown below.

assignment ::= variable_lvalue = [delay_or_event_control] expression

The left-hand side of a procedural assignment *<lvalue>* can be one of the following:

- A reg, integer, real, or time register variable or a memory element
- A bit select of these variables (e.g., addr[0])
- A part select of these variables (e.g., addr[31:16])
- A concatenation of any of the above

The right-hand side can be any expression that evaluates to a value. In behavioral modeling, all operators discussed in Dataflow Modeling can be used in behavioral expressions.

12.1.3 Timing Controls

Various behavioral timing control constructs are available in Verilog. In Verilog, if there are no timing control statements, the simulation time does not advance. Timing controls provide a way to specify the simulation time at which procedural statements will execute. There are three methods of timing control:

- delay-based timing control
- event-based timing control
- level-sensitive timing control

Delay-Based Timing Control

Delay-based timing control in an expression specifies the time duration between when the statement is encountered and when it is executed. Delay-based timing control examples can be seen in the previous sections. There are three types of delay control for procedural assignments:

- regular delay control
- intra-assignment delay control
- zero delay control

Event-Based Timing Control

An event is the change in the value on a register or a net. Events can be utilized to trigger execution of a statement or a block of statements. There are four types of event-based timing control:

- regular event control
- named event control
- event OR control
- level-sensitive timing control

Regular event control

The @ symbol is used to specify an event control. Statements can be executed on changes in signal value or at a positive or negative transition of the signal value. The keyword posedge is used for a positive transition, as shown in example.

Example
@(clock) q = d; //**q = d is executed whenever signal clock changes value**
@(posedge clock) q = d; //**q = d is executed whenever signal clock does a positive transition**

i. Event OR Control

Sometimes a transition on any one of multiple signals or events can trigger the execution of a statement or a block of statements. This is expressed as an OR of events or signals. The list of events or signals expressed as an OR is also known as a sensitivity list. The keyword or is used to specify multiple triggers, as shown in Example.

Example

```
always @( reset or clock or d) //Wait for reset or clock or d to change
begin
   if (reset) //if reset signal is high, set q to 0
        q = 1'b0;
          else if(clock) //if clock is high, q = d
            q = d;
end
```

Sensitivity lists can also be specified using the "," (comma) operator instead of the or operator. Comma operators can also be applied to sensitivity lists that have edge-sensitive triggers. The above example can be rewritten using the comma operator, the only change would be:

always @(reset, clock, d)

When the number of input variables to a combination logic block are very large, sensitivity lists can become very cumbersome to write. To solve this problem, Verilog HDL contains two special symbols: @* and @(*). Both symbols exhibit identical behavior. These special symbols are sensitive to a change on any signal that may be read by the statement group that follows this

symbol.

```
//Combination logic block using the or operator
//Cumbersome to write and it is easy to miss one input to the block
always @(a or b or c or d or e)

begin
out1 = a ? b+c : d+e;
 end

//Instead of the above method, use @(*) symbol. Alternately, the @* symbol can be used
//All input variables are automatically included in the sensitivity list

always @(*)
begin
out1 = a ? b+c : d+e;
end
```

Level-Sensitive Timing Control

Event control discussed earlier waited for the change of a signal value or the triggering of an event. The symbol @ provided edge-sensitive control. Verilog also allows level-sensitive timing control, that is, the ability to wait for a certain condition to be true before a statement or a block of statements is executed. The keyword wait is used for level-sensitive constructs.

12.1.4 Conditional Statements

Conditional statements are used for making decisions based upon certain conditions. These conditions are used to decide whether or not a statement should be executed. Keywords if and else are used for conditional statements. There are three types of conditional statements. Usage of conditional statements is shown below.

```
//Type 1 conditional statement. No else statement
//Statement executes or does not execute
if (<expression>) true_statement ;

//Type 2 conditional statement. One else statement
//Either true_statement or false_statement is evaluated
if (<expression>) true_statement ; else false_statement ;

//Type 3 conditional statement. Nested if-else-if
//Choice of multiple statements. Only one is executed.
if (<expression1>) true_statement1 ;
else if (<expression2>) true_statement2 ;
else if (<expression3>) true_statement3 ;
else default_statement ;
```

The *<expression>* is evaluated. If it is true (1 or a non-zero value), the *true_statement* is executed. However, if it is false (zero) or ambiguous (x), the *false_statement* is executed. The *<expression>* can contain any operators used in dataflow modeling. Each *true_statement* or *false_statement* can be a single statement or a block of multiple statements. A block must be grouped, typically by using keywords **begin** and **end**. A single statement need not be grouped.

12.1.5 Multiway Branching

In type 3 conditional statement in the previous section, there were many alternatives, from which one was chosen. The nested if-else-if can become unwieldy if there are too many alternatives. A shortcut to achieve the same result is to use the case statement.
The keywords case, endcase, and default are used in the case statement.

```
case (expression) alternative1: statement1;
        alternative2: statement2;
        alternative3: statement3;
        ...
        ...
       default: default_statement;
endcase
```

Each of *statement1, statement2 …, default_statement* can be a single statement or a block of multiple statements. A block of multiple statements must be grouped by keywords **begin** and **end**. The expression is compared to the alternatives in the order they are written. For the first alternative that matches, the corresponding statement or block is executed. If none of the alternatives matches, the *default_statement* is executed. The *default_statement* is optional. Placing of multiple default statements in one case statement is not allowed. The case statements can be nested. The case statement can also act like a many-to-one multiplexer

Example: 4-to-1 Multiplexer with Case Statement

```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);
// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3; input s1, s0;
reg out;

always @(s1 or s0 or i0 or i1 or i2 or i3)
case ({s1, s0}) //Switch based on concatenation of control signals
        2'd0 : out = i0;
        2'd1 : out = i1;
        2'd2 : out = i2; 2'd3 :
        out = i3;
        default: $display("Invalid control signals");
endcase

endmodule
```

The case statement compares 0, 1, x, and z values in the expression and the alternative bit for

bit. If the expression and the alternative are of unequal bit width, they are zero filled to match the bit width of the widest of the expression and the alternative.

12.2 Verilog Codes and task:

Half Adder

```
module half_adder(S, C, A, B);

output S, C;

reg S, C; //output must be declared as register in behavioral modeling
input A, B; //always block makes sure that whenever is a change in A or B, we enter this block

always @ (A or B)

begin
        {C,S} = A+B;
end

endmodule
```

Main Module (Full Adder with instantiation)

```
module full_adder(Sum, CarryOut, A, B, CarryIn);

output Sum, CarryOut;
input A, B, CarryIn;
wire TSum, TCarry1, TCarry2;

//Two half-adders are instantiated. (This is similar to a function call in C)
half_adder ha1(TSum, TCarry1, A, B);
half_adder ha2(Sum, TCarry2, TSum, CarryIn);

//Or gate can be written as this assign statement
assign CarryOut = TCarry1 || TCarry2;

endmodule
```

Main Module (Full Adder without instantiation)

```
module full_adder(Sum, CarryOut, A, B, CarryIn);

output Sum, CarryOut;
reg Sum, CarryOut; //output must be declared as register in behavioral modeling
input A, B, CarryIn;
```

```
always @ (A, B, CarryIn) begin
{CarryOut,Sum} = A+B+CarryIn;
end

endmodule
```

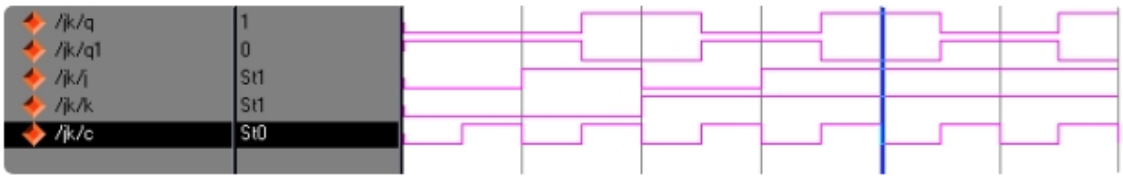
Stimulus

```
module Stimulus;

wire Sum, CarryOut;
reg A, B, CarryIn;

//This time we have instantiated full-adder module as it is the one we are testing
full_adder fa1(Sum, CarryOut, A, B, CarryIn);

//Values checking part
Initial
 begin
          A = 0; B = 0; CarryIn = 0; //Sum = 0, CarryOut = 0
#10      A = 0; B = 0; CarryIn = 1; //Sum = 1, CarryOut = 0
#10      A = 0; B = 1; CarryIn = 0; //Sum = 1, CarryOut = 0
#10      A = 0; B = 1; CarryIn = 1; //Sum = 0, CarryOut = 1
#10      A = 1; B = 0; CarryIn = 0; //Sum = 1, CarryOut = 0
#10     A = 1; B = 0; CarryIn = 1; //Sum = 0, CarryOut = 1
#10     A = 1; B = 1; CarryIn = 0; //Sum = 0, CarryOut = 1
#10     A = 1; B = 1; CarryIn = 1; //Sum = 1, CarryOut = 1
#10 $stop;
#10 $finish;
end
endmodule
```

Verilog code for jk flip flop

```
module jk(q,q1,j,k,c);
output q,q1;
input j,k,c;
reg q,q1;
initial begin q=1'b0; q1=1'b1; end
always @ (posedge c)
 begin
          case({j,k})
                    {1'b0,1'b0}:begin q=q; q1=q1; end
                    {1'b0,1'b1}: begin q=1'b0; q1=1'b1; end
                    {1'b1,1'b0}:begin q=1'b1; q1=1'b0; end
                    {1'b1,1'b1}: begin q=~q; q1=~q1; end
          endcase
```

end
endmodule



Simulated waveform for J-K Flip Flop

Lab Task:

Lab Tasks will be provided at the end of this lab by the instructor.

Lab - 13: Open Ended Lab

13.1 Objective

To make a joint project within the lab addressing a real life problem using Xilinx ISE. An open ended lab is where students are given the freedom to develop their own critical thinking to solve a given problem, instead of merely following the preset guidelines from the lab manual or a book.

13.2 Pre-Lab Reading

Free to use knowledge from the previous labs. The instructor may drop pointers for the students and perhaps talk about the difficulties that the student face while solving the problem. This may be in the form of a lecture or a discussion and emphasis on essential learning points.

13.3 Activities and Exercise

The problem statement will be provided by the instructor before the start of the open ended lab.

LEVEL	PROBLEM	WAYS & MEANS	ANSWERS
0	Given	Given	Given
1	**Given**	**Given**	**Open**
2	Given	Open	Open
3	Open	Open	Open

Table 13.1. Schwab/Herron Levels of Laboratory Openness

Level 1 of Schwab/Herron Laboratory Openness will be followed and introduced in the manual.